



**JOMO KENYATTA UNIVERSITY OF AGRICULTURE AND
TECHNOLOGY**
**SCHOOL OF ELECTRICAL, ELECTRONIC AND INFORMATION
ENGINEERING**
DEPARTMENT OF ELECTRONIC AND COMPUTER ENGINEERING
FINAL YEAR PROJECT PROPOSAL

STUDENT DETAILS

<u>NAME</u>	<u>ADMISSION</u>
KIOGORA IAN MWENDA	ENE212-0251/2021

COURSE: B.Sc. ELECTRONIC AND COMPUTER ENGINEERING
FINAL YEAR PROJECT
PROJECT PROPOSAL
**TITLE: VIRTUALIZED AR WINDSCREEN DISPLAY WITH SEMI-
AUTONOMOUS DRIVING SUPPORT SYSTEM**

LECTURER: MR. OMBATI
DATE OF SUBMISSION: 01/10/2026

Abbreviations and Acronyms

Abbreviation	Meaning
ADAS	Advanced Driver-Assistance System
AEB	Autonomous Emergency Braking
AR	Augmented Reality
CAN	Controller Area Network (in-vehicle bus)
CPA	Closest Point of Approach
CRC	Cyclic Redundancy Check
CTRV	Constant Turn Rate and Velocity (motion model)
CUDA	Compute Unified Device Architecture (NVIDIA GPU computing platform)
CV	Constant Velocity (motion model)
EKF	Extended Kalman Filter
FCW	Forward Collision Warning
FreeRTOS	Free Real-Time Operating System
GNSS / GPS	Global Navigation Satellite System / Global Positioning System
GPU	Graphics Processing Unit
HMM	Hidden Markov Model
HUD	Head-Up Display
I2C	Inter-Integrated Circuit (serial bus)
IMM	Interacting Multiple Model (filter)
IMU	Inertial Measurement Unit
IWDG	Independent Watchdog (STM32 hardware timer)
LiDAR	Light Detection and Ranging
mAP	mean Average Precision
MLD	Multi-Level Dijkstra (OSRM routing algorithm)
MTSD	Mapillary Traffic Sign Dataset
NTSA	National Transport and Safety Authority (Kenya)
OBD-II	On-Board Diagnostics, second generation
OSM	OpenStreetMap
OSRM	Open Source Routing Machine
PCB	Printed Circuit Board
PWM	Pulse-Width Modulation
TensorRT	NVIDIA's deep-learning inference optimiser and runtime
TTC	Time to Collision
UART	Universal Asynchronous Receiver-Transmitter
USB-CDC	USB Communications Device Class (virtual serial port)
WSL2	Windows Subsystem for Linux, version 2
YOLO	You Only Look Once (single-stage object detector family)

Table of Contents

Abstract	
1.0 Introduction and Background	
1.1 Problem Statement	
1.2 Problem Justification	
1.3 Objectives	
1.4 Scope and Limitations	
2.0 Literature Review	
2.1 Road Safety, Driver Attention and Navigation Displays	
2.2 Augmented-Reality Head-Up Displays	
2.3 Camera-Based Perception with Deep Learning	
2.4 Camera–LiDAR Fusion, State Estimation and Tracking	
2.5 Routing and Map Matching	
2.6 Collision Avoidance and Functional Safety	
2.7 Summary of Gaps and the Proposed Improvements	
3.0 Methodology	
3.1 Development Approach	
3.2 System Architecture	
3.3 Hardware Design: the Two-Box Hub	
3.4 Host–Hub Protocol and the Safety Core	
3.5 Perception	
3.6 Sensor Fusion, Tracking and Motion Prediction	
3.7 Precision Navigation	
3.8 Hazard Assessment and the Decision Arbiter	
3.9 AR Rendering and Display	
3.10 Calibration	
3.11 Testing and Staged Validation	
3.12 Demonstration and Presentation Plan	
4.0 Expected Results	
5.0 Project Time Plan	
6.0 Budget Estimates	
References	
Appendix A: Component Specifications	
Appendix B: Host–Hub Protocol Reference	
Appendix C: Safety Properties and Fault Codes	
Appendix D: Key Algorithms	
Appendix E: Test Plan and Checklists	
Appendix F: Development Status and Verification to Date	

Abstract

Road crashes killed an estimated 1.19 million people worldwide in 2021 [1], and Kenya records over four thousand road deaths a year, most of them pedestrians and motorcyclists [2]. Many of these crashes begin with a driver who looked away at the wrong moment: at a phone used for navigation, or at a hazard noticed too late. Driver-assistance technology that addresses this, such as augmented-reality (AR) head-up displays and automatic emergency braking, is sold almost exclusively as factory equipment in new, expensive vehicles, and none of it can be fitted to the older cars that make up most of Kenya's fleet.

This project proposes a **Virtualized AR Windscreen Display with Semi-Autonomous Driving Support System**: a retrofit platform that fuses a 2K camera, a 360° LiDAR, satellite positioning, an inertial sensor and the car's own OBD-II data into a real-time model of the road ahead. The model drives two things. First, a **precision AR navigation display**, in which the route is drawn onto the live camera view locked to the measured road surface and to the correct lane, rather than as a generic arrow floating over the image. Second, a **semi-autonomous safety layer** that detects and tracks road users, warns of hazards and reckless driving, operates the indicators and hazard lights, and, as a last resort, applies a force-limited brake through a fail-safe actuator which the driver can always override.

The system is split between a laptop with an NVIDIA RTX 5060 GPU, which runs four TensorRT neural networks (object segmentation, Kenyan road-sign detection, lane detection and depth), sensor fusion, tracking, offline routing for a 100 km region around Nairobi and GPU rendering; and an STM32 microcontroller hub running FreeRTOS, built on two custom circuit boards: a windscreen pod behind the mirror carrying the camera and inertial sensor, and an under-dash power box whose outputs all default to "off" in hardware. The hub owns every actuator, reads the car's own CAN bus, and enforces the safety rules independently of the laptop, including release within 200 ms if the laptop stops responding. Much of the software has already been built and tested against recorded road video and a recorded Nairobi drive: the four networks run together in a median 17.8 ms per frame against a 33 ms budget, the AR renderer runs at 54–82 frames per second at 2560×1440, and the braking logic fires at the intended time-to-collision in simulation. The remaining work is hardware integration, calibration, a mandatory actuator bench test, and staged validation from a bench, to a stationary car, to an empty field, to a closed campus road at 5–10 km/h. The estimated hardware budget is KES 195,000–270,000, most of which is the LiDAR.

Keywords: augmented reality, head-up display, driver assistance, sensor fusion, LiDAR, deep learning, map matching, emergency braking, functional safety, embedded systems, CAN bus.

1.0 Introduction and Background

Driving is mostly a visual task. A driver continuously reads the road ahead, the vehicles and people around them, the signs, and, increasingly, a navigation screen mounted on the dashboard or held in the hand. Every glance away from the road is a period in which the driver is effectively blind to what happens in front of the car, and naturalistic driving studies have repeatedly found that such glances are among the strongest predictors of a crash [3][4].

Two families of technology have grown up to address this. The first is the **head-up display (HUD)**, which places information in the driver's line of sight so that the eyes do not leave the road. Its most advanced form, the **augmented-reality (AR) HUD**, goes further: it draws graphics that appear attached to the real world, such as a navigation path lying on the correct lane or a highlight around a pedestrian, so that the driver does not need to translate a map or an abstract symbol into the scene in front of them [5][6]. The second family is **advanced driver-assistance systems (ADAS)**: forward collision warning, lane keeping, and autonomous emergency braking (AEB), which use cameras and radar to detect danger and, in the case of AEB, brake the car when the driver does not. Large studies of AEB have found it reduces rear-end crashes substantially [7].

Both families share a limitation that matters in Kenya: they are factory-fitted to new, mid-range and premium vehicles and cannot be added afterwards. Commercial AR windscreen products, such as those demonstrated by WayRay and by vehicle manufacturers, are proprietary, costly and tied to specific car models. Most cars on Kenyan roads are older imports that will never receive them. At the same time, the components from which such a system is built (cameras, compact LiDAR sensors, satellite receivers, inertial sensors, consumer GPUs that can run neural networks in real time, and inexpensive microcontrollers) have become affordable and widely available.

This project proposes to combine those components into a **retrofit** platform: a **Virtualized AR Windscreen Display with Semi-Autonomous Driving Support System**. It is “virtualized” because, instead of an optical combiner projected on the glass, the first version shows the AR view on a screen as a live camera image with the graphics composited into it (video see-through AR), while the software is structured so that an optical display can replace the screen later without changing anything else. It is “semi-autonomous” because the car never drives itself: the system informs the driver, operates the indicators, hazard lights and horn, and, only when a collision is imminent and the driver has not reacted, applies a limited brake that the driver can always overpower.

The central idea is that a single, precise, real-time model of the road ahead serves both purposes. The same sensor fusion that tells the navigation display exactly where the road surface and the lane are also tells the safety layer where every vehicle, cyclist and pedestrian is, how fast they are moving and where they are likely to go next.

1.1 Problem Statement

Kenya's National Transport and Safety Authority recorded 4,690 road deaths in 2022, of whom 1,682 were pedestrians and 1,254 were motorcyclists [2]. Globally the World Health Organization estimates 1.19 million deaths in 2021, with more than nine in ten occurring in low- and middle-income countries [1]. Three driver-side problems contribute to this toll and are the focus of this project:

1. **Delayed recognition of hazards.** Pedestrians crossing between vehicles, motorcycles filtering through traffic, vehicles braking suddenly and vehicles driven recklessly are recognised late, especially at night, in rain and in dense urban traffic. The time the driver has left to react is often shorter than their reaction time.
2. **Distraction caused by navigation.** Drivers who do not know the route glance repeatedly at a phone or a dashboard screen and must mentally map a two-dimensional arrow onto a complex junction. Each glance removes their eyes from the road; the 100-car naturalistic study found that glances away from the forward roadway of more than two seconds roughly doubled near-crash and crash risk [3]. Kenyan roads add a further difficulty: road works, closures and new roads change the network faster than a static map can follow.
3. **No automatic protective response.** In the vehicles most Kenyans drive, nothing reacts on the driver's behalf when a collision becomes imminent: no warning, no hazard lights to alert following traffic, no emergency braking.

Solutions to each problem exist, but only as factory-fitted, model-specific and expensive equipment. There is therefore no affordable, retrofittable system that gives the driver of an ordinary car precise, eyes-on-the-road navigation and hazard awareness, together with a safe, bounded automatic response when the driver does not react in time.

1.2 Problem Justification

Solving this problem is worthwhile for four reasons.

The need is large and local. The groups most at risk on Kenyan roads, pedestrians and motorcyclists, are exactly those a camera–LiDAR perception system can detect and highlight. Even a modest reduction in late recognition of such road users would have real value.

The evidence supports the approach. AR navigation that places guidance on the road reduces the need to look away, and drivers using it have reported clear benefits over a conventional display [6], provided the graphics are designed not to become a distraction themselves [8]. Forward collision warning combined with automatic braking has been associated with a large reduction in front-to-rear crashes with injuries [7].

A retrofit is now technically and economically feasible. A consumer laptop GPU can run several neural networks on high-resolution video in real time, a compact LiDAR costs a fraction of earlier automotive units, and open data and software (OpenStreetMap, OSRM, TensorRT, FreeRTOS) remove licensing costs. The whole system is estimated at KES 195,000–270,000, far below the cost of a vehicle equipped with comparable factory features.

It is a substantial engineering project in the department's core areas. It spans embedded systems and real-time firmware, power and actuator electronics, communication protocols, digital signal processing and estimation, computer vision and machine learning, computer graphics, and safety engineering, which is the breadth expected of a final year project in Electronic and Computer Engineering.

1.3 Objectives

1.3.1 Main Objective

To design, implement and test a precision, sensor-fused augmented-reality navigation and driver support system, whose real-time model of the road also drives hazard detection and fail-safe, driver-overridable vehicle actuation, as a retrofit for an ordinary car.

1.3.2 Specific Objectives

1. To design and implement a real-time perception and sensor-fusion pipeline that detects, segments and ranges vehicles, pedestrians, cyclists, lanes and Kenyan road signs from camera and LiDAR data, and estimates the car's own motion from GNSS, IMU and OBD-II, processing 2K video at 30 frames per second (all neural networks within a 33 ms frame budget) with a road-sign detection accuracy of at least 0.70 mAP50.
2. To design and implement a road-locked AR navigation display that routes offline across a 100 km region around Nairobi, matches the car's position to the road, and draws the route in the correct lane on the measured road surface at 24 frames per second or more, with the drawn path within 2 pixels of the calibrated camera model and visibly inside the correct lane within LiDAR range.
3. To design, implement and validate a hazard-assessment and fail-safe actuation subsystem that warns of hazards and reckless driving, operates indicators, hazard lights and horn, and brakes through a force-limited actuator only when time-to-collision falls below 1.8 s and the driver has not braked, verified by passing all six actuator bench checks on three consecutive attempts (including driver override, release within 200 ms of link loss, and release when the power box is disconnected) before staged in-vehicle testing at 5–10 km/h.

1.4 Scope and Limitations

In scope. The full sensing, perception, fusion, tracking, navigation, decision and rendering software on the laptop; the sensor and actuator hub (firmware, and two custom PCBs with their enclosures: a windscreen pod and an under-dash power box); a roof mount for the LiDAR; a force-limited brake actuator with a fail-safe release; a physical emergency-stop switch; a bench rig for the actuator; installation in one test vehicle; and staged validation up to a closed campus road section at 5–10 km/h.

Out of scope for this project, and presented as future work: an optical combiner or projector on the windscreen (the software's display interface is designed so this can be added without other changes); steering intervention of any kind; fully autonomous driving; thermal cameras; and cloud connectivity or mobile applications.

Limitations. Navigation precision is highest within LiDAR range, where the road surface is measured, and falls back to a flat-road assumption beyond it; this will be reported as it is. Testing is limited to low speeds on closed or empty ground, so stopping-distance calibration at road speeds is not claimed. A live road-status layer (traffic and incidents) depends on a commercial data provider's licence terms and is deferred; the system routes on an offline OpenStreetMap graph that is refreshed regularly and corrected by the car's own sensors.

2.0 Literature Review

This chapter reviews the work on which each part of the proposed system builds: why driver attention and navigation matter (2.1); augmented-reality head-up displays (2.2); deep-learning perception from a camera (2.3); fusing the camera with LiDAR and estimating motion (2.4); routing and map matching (2.5); and collision avoidance and functional safety (2.6). Each section ends by stating what this project takes from the literature and where it differs. Section 2.7 draws the gaps together.

2.1 Road Safety, Driver Attention and Navigation Displays

The WHO's 2023 global status report estimates 1.19 million road deaths in 2021, a slight fall from 2010 but still the leading cause of death for people aged 5–29, with more than 90% of deaths in low- and middle-income countries and more than half among pedestrians, cyclists and motorcyclists [1]. Kenyan figures follow the same pattern: pedestrians and motorcyclists together made up more than 60% of the 4,690 deaths recorded in 2022 [2]. Any system meant for Kenyan roads must therefore detect vulnerable road users reliably, not only other cars.

The link between inattention and crashes is well established by *naturalistic driving studies*, in which ordinary drivers' cars are instrumented with cameras and sensors for months. Klauer *et al.* analysed the 100-car study and found that any glance away from the forward roadway lasting more than two seconds at least doubled the risk of a crash or near-crash, and that secondary tasks were a factor in a large share of events [3]. Dingus *et al.*, using the much larger SHRP 2 dataset of over 900 crashes, found that drivers were engaged in some observable distracting activity during more than half of their driving time, and that handling a hand-held phone raised crash risk several-fold [4].

These studies show why a conventional navigation display is itself part of the problem: to use it, the driver must look down, read an abstract map, and translate it back to the scene. Placing guidance in the driver's line of sight and attaching it to the road is the obvious response, and leads to head-up and AR displays.

2.2 Augmented-Reality Head-Up Displays

Gabbard, Fitch and Kim reviewed the prospects for automotive AR and set out both its promise and its difficulties [5]. The promise is that graphics *registered* to the world (appearing to lie at the correct place in the scene) remove the mental translation step: a turn arrow painted on the correct lane needs no interpretation. The difficulties are technical and human. Registration must be accurate and stable, or the graphics drift, jitter or appear to float, which is worse than no graphics at all. The graphics must not hide real hazards. And the system's latency must be low enough that the overlay keeps up with the car's motion.

Bark *et al.* built "Personal Navi", a navigation aid on a see-through 3D volumetric HUD, and compared AR guidance that appeared at the depth of the road with conventional guidance in a driving study [6]. The study reported advantages for guidance presented at the depth of the road, supporting the idea that guidance attached to the road reduces the effort of following a route. Their system, however, relied on a specialised optical display and on a pre-defined route geometry, not on sensing the road itself.

The risk of AR adding distraction rather than removing it is addressed by Kim and Gabbard, who measured how different AR HUD designs affect drivers’ visual attention [8]. They found that the distraction potential depends strongly on how the graphics are designed and placed, and that it can and should be measured rather than assumed. This project turns that finding into design rules: one consistent visual language, graphics that stay on the road surface, graphics shown only when relevant, and no overlay covering a hazard.

Commercial systems follow the optical route. Production AR HUDs from vehicle manufacturers, and holographic windscreen displays shown by companies such as WayRay, project graphics through the windscreen with an optical combiner. They are engineered for a specific car’s windscreen and dashboard, are proprietary, and are not available as retrofits.

Relevance to this project. The proposal adopts the AR principle and the design rules from [5][8], but makes two different choices. First, it uses *video see-through* AR on a screen for the first version, which removes the optical engineering while keeping the registration problem, the hard part. Second, the registration is driven by *measurement*: the route is placed using the LiDAR-measured road surface and the detected lane lines, rather than a flat-road assumption or pre-defined geometry.

2.3 Camera-Based Perception with Deep Learning

Object detection and segmentation. Redmon *et al.* introduced YOLO, which treats detection as a single regression from the image to boxes and class probabilities and so runs in real time, unlike earlier two-stage detectors [9]. The family has since gone through many versions; Terven, Córdova-Esparza and Romero-González review them up to YOLOv8, whose anchor-free head and improved training give a good balance of speed and accuracy [10]. Bolya *et al.* showed with YOLACT that *instance segmentation*, a pixel-accurate mask for each object, can also run in real time, by predicting a small set of prototype masks per image and a set of coefficients per object [11]. YOLOv8’s segmentation variant uses the same idea. Masks matter for this project for two reasons: the AR highlight can follow a pedestrian’s outline, and, more importantly, the mask selects exactly which LiDAR points belong to the object (Section 2.4).

Lane detection. Qin, Wang and Li reformulated lane detection as selecting, for each of a set of predefined image rows, which column the lane passes through, rather than classifying every pixel [12]. This “row anchor” approach is far faster than segmentation and copes with lanes hidden by vehicles because it reasons about the whole image. Their follow-up, UFLDv2, adds column anchors and an ordinal classification loss, improving accuracy on curved and side lanes while keeping the speed [13].

Monocular depth. Ranftl *et al.* trained MiDaS on a mixture of datasets with a scale- and shift-invariant loss, producing a network that predicts *relative* depth for almost any image [14]. Its output has no metric scale, so it cannot on its own give a distance in metres, but it is useful for ordering objects and, when scaled against a few measured points, for estimating the range of objects the LiDAR does not see.

Traffic signs. Ertler *et al.* released the Mapillary Traffic Sign Dataset (MTSD), with over 100,000 street-level images from around the world and more than 300 sign classes [15]. Its global coverage makes it a better starting point for Kenyan roads than datasets collected in one European country, since Kenyan signs follow the Vienna-convention style with local variations.

Relevance to this project. Detection alone gives *what* and *where in the image*, not *how far*. Most published systems evaluate one network at a time on a desktop GPU. This project runs four networks together (YOLOv8m-seg, a YOLOv8 sign detector trained on MTSD and audited on Kenyan images, UFLDV2 and MiDaS) within one frame period on a laptop GPU, and uses each network’s output as an input to geometry and fusion rather than as an end result.

2.4 Camera–LiDAR Fusion, State Estimation and Tracking

Geometry and calibration. Fusing a camera with any 3D sensor starts from the pinhole camera model and its lens distortion, which map a 3D point to a pixel [16]. Zhang’s method estimates these parameters from several views of a flat checkerboard and is the standard procedure for camera calibration [17]. The rigid transform between LiDAR and camera (the *extrinsics*) must also be known; small errors in it move projected points off the objects they belong to.

Fusion. Qi *et al.*’s Frustum PointNets use a 2D detection to cut out the frustum of 3D points behind each box and then find the object inside it with a learned network [18]. Vora *et al.*’s PointPainting goes the other way: it projects every LiDAR point into a semantic segmentation of the image and “paints” it with the class scores before 3D detection [19]. Both show that the camera is best at deciding *what* something is and the LiDAR at measuring *where* it is. Both, however, rely on trained 3D networks and on large labelled LiDAR datasets. Box-based selection also has a known weakness: a pedestrian’s bounding box is mostly background, so points from the wall behind them contaminate the range.

State estimation. The Kalman filter [20] is the optimal estimator for linear systems with Gaussian noise; the extended Kalman filter (EKF) linearises a nonlinear model around the current estimate, and the unscented filter of Julier and Uhlmann propagates sample points instead, which handles stronger nonlinearity [21]. Thrun, Burgard and Fox give the probabilistic framework, including motion models for vehicles and the fusion of satellite, inertial and odometry measurements [22].

Tracking manoeuvring targets. A single motion model cannot describe a car that cruises, turns and brakes. Blom and Bar-Shalom’s *interacting multiple model* (IMM) filter runs several models in parallel (for example constant velocity, constant turn rate, and stopping) and mixes them according to how well each explains recent measurements [23]. Tracking many objects adds the *data association* problem, deciding which detection belongs to which track, which is commonly solved as an assignment problem with the Hungarian algorithm [24] inside validation gates [25].

Relevance to this project. The proposal fuses with *instance masks* rather than boxes: only points inside an eroded mask are assigned to an object, and the range is taken from the nearest dominant cluster of those points, never their mean. This needs no trained 3D network and removes background contamination at its source. Tracking uses an IMM filter with constant velocity, constant turn rate and velocity (CTRV) and stopping models, which also provides a per-object probability that a vehicle ahead is braking.

2.5 Routing and Map Matching

Luxen and Vetter presented OSRM, an open-source routing engine for OpenStreetMap data that answers continental-scale route queries in milliseconds by preprocessing the road graph [26]. Delling *et al.*’s *customizable route planning* separates that preprocessing into a metric-independent partition of

the graph and a fast customisation step that applies road weights [27]. OSRM’s multi-level Dijkstra (MLD) mode implements this idea, which means that closures or traffic can re-weight the graph in seconds without rebuilding it.

Raw GNSS positions are typically in error by several metres and cannot say which road, let alone which lane, the car is on. Newson and Krumm formulated *map matching* as a hidden Markov model: candidate road positions near each fix are hidden states, emission probabilities fall with distance from the fix, and transition probabilities favour routes whose driven length matches the straight-line distance between fixes; the Viterbi algorithm then finds the most likely path [28]. This is robust to noise and to sparse fixes and is the basis of OSRM’s own matching service.

Relevance to this project. Routing and map matching establish *which road* and *how far along it* the car is, but not where the road is in the camera image with lane-level precision. The proposal therefore adds a step not found in these works: a projector that places the matched route on the LiDAR-measured road surface and corrects its heading and lateral position from the detected lane lines.

2.6 Collision Avoidance and Functional Safety

Lee showed that drivers regulate braking using *time to collision* (TTC), the current gap divided by the closing speed, which can be perceived directly from how fast an object’s image grows [29]. TTC has since become the standard trigger quantity in forward collision warning and AEB. Cicchino’s analysis of police-reported crashes in the United States found that vehicles with forward collision warning plus low-speed AEB had about half the rate of front-to-rear crashes of the same models without them [7].

A system that can apply the brakes is safety-critical. ISO 26262 sets out the functional-safety lifecycle for road-vehicle electronics: hazard analysis, safety goals, a safe state for every fault, independence between the function and its monitor, and verification evidence at each stage [30]. Two concerns stand out for a retrofit. The first is that a false activation, *phantom braking*, is itself a hazard. The second is that software on a general-purpose computer cannot be trusted to guarantee a safe state.

Relevance to this project. The proposal does not claim ISO 26262 compliance, which is beyond a student project, but adopts its principles: a defined safe state (brake released), an independent monitor (the hub, which enforces limits and releases on its own watchdog), a hardware kill path (the emergency stop), and an evidence log of every decision. To limit phantom braking, braking is triggered only by the measured TTC of a confirmed, LiDAR-ranged object, never by a prediction or a learned model, as recommended by the TTC literature [29].

2.7 Summary of Gaps and the Proposed Improvements

Table 1 compares the reviewed work with the proposed system. The works reviewed each solve one part of the problem well; none combines precise, measured AR registration with perception, tracking and a fail-safe actuation path in a system that can be retrofitted to an ordinary car.

Table 1: Comparison of existing approaches with the proposed system

Approach	What it does well	Gap for this problem	How this project differs
Commercial AR HUDs (factory, optical)	High-quality optical AR in the driver’s line of sight	Proprietary, expensive, model-specific; not retrofittable	Retrofit hardware; video see-through first, optical display interface designed in

AR navigation studies [6][8]	Show the benefit and the design rules of conformal graphics	Route geometry assumed or pre-defined; no sensing of the road	Route placed on the LiDAR-measured surface, corrected by detected lanes
Real-time detectors [9][11][13]	Fast, accurate image-space detection	No metric range; evaluated in isolation	Four networks together within one frame; outputs feed geometry and fusion
Learned camera–LiDAR fusion [18] [19]	Accurate 3D detection	Needs trained 3D networks and labelled LiDAR data; box-based selection mixes in background	Mask-based point assignment with nearest-cluster range; no 3D training needed
Routing and HMM map matching [26] [28]	Which road, how far along it	Metre-level error; no lane or surface information	Adds the road-surface projector; three road-status layers where sensors override the map
Factory FCW/AEB [7]	Proven crash reduction	Factory-only; closed design	Retrofit, force-limited, driver-overridable, with an open, auditable trigger rule and an evidence log

In summary, the improvements this project makes over existing work are: (i) **measured registration** of the AR route, from the combination of LiDAR ground model, lane detection and map matching; (ii) **mask-based fusion** that gives a reliable range to each detected object without training a 3D network; (iii) **one road model serving both navigation and safety**; and (iv) a **retrofit, fail-safe actuation path** in which an independent microcontroller, a hardware kill switch and a mechanical force limit each keep the brake safe even if the laptop fails.

3.0 Methodology

This chapter describes how the system will be built and verified: the development approach (3.1), the overall architecture (3.2), the hardware and firmware of the hub (3.3–3.4), each software subsystem on the laptop (3.5–3.9), calibration (3.10), testing and staged validation (3.11), and how the work will be demonstrated (3.12). Design calculations are given where a component or threshold is chosen.

3.1 Development Approach

The project is developed in seventeen phases (0–16), each with written exit criteria that must be demonstrably met before the next phase begins (Figure 1). Each phase starts with the concept and the interface, then the implementation, then tests, including tests designed to fail if the code is wrong (for example, a projection test that fails if lens distortion is left out). Software that does not need hardware is developed and verified first against recorded road video, a recorded GNSS drive and synthetic scenes, so that when the hardware arrives, only integration remains. Every task is recorded in a progress log (what was done, why, and how it was verified), and every choice made under ambiguity is recorded in a decision log, which together form the evidence base for the final report.

Hardware work follows a strict order with a mandatory gate: **no part of the system may act on a real vehicle until the brake actuator has passed its bench test** (Section 3.11). All physical work (wiring, mounting, testing in the car) is carried out by the student.

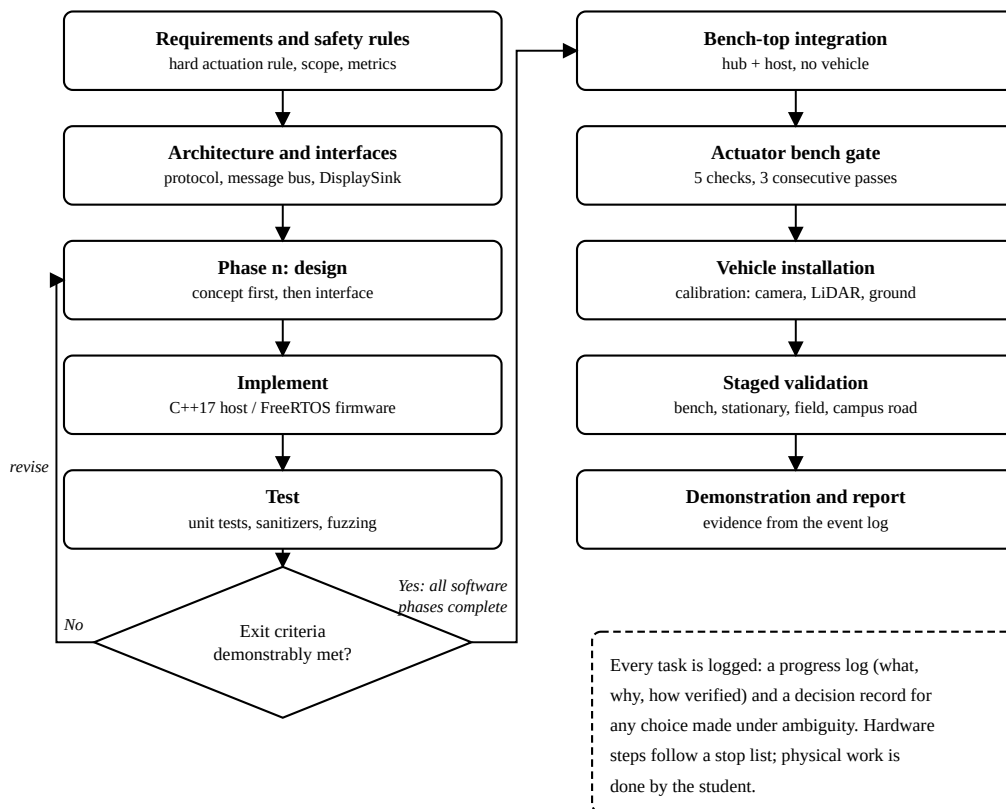


Figure 1: Development methodology: phased design with exit gates, followed by the hardware integration and staged validation sequence

3.2 System Architecture

The system has four layers (Figure 2). **Sensors** observe the road and the car. A **sensor and actuator hub**, an STM32F405 microcontroller running FreeRTOS, reads the low-rate sensors (GNSS, IMU, OBD-II over the car’s CAN bus, gesture), owns every actuator, and enforces the safety rules. The **host**, a laptop with an NVIDIA RTX 5060 GPU running Debian Linux under WSL2, reads the high-rate sensors (camera over USB, LiDAR over Ethernet) and runs perception, fusion, navigation, decision-making and rendering. The **outputs** are the AR display, the signal relays, the brake actuator, and an evidence log.

The split follows one principle: *the laptop decides, the hub disposes*. The laptop has the computing power for vision and mapping but runs a general-purpose operating system that can crash or stall. The hub is simple, deterministic and always running. The laptop may only send a bounded *request*; the hub decides whether to act on it, clamps it to its own limits, and releases the brake by itself if the laptop falls silent for 200 ms.

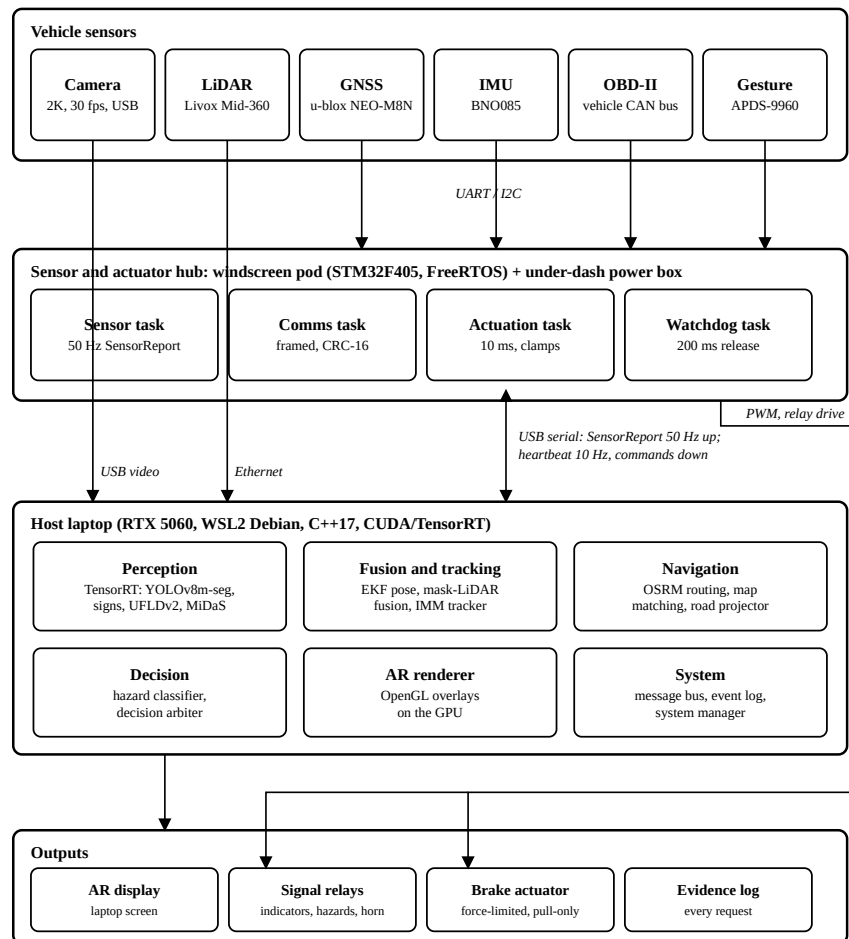


Figure 2: System architecture, from sensors to outputs

Physical layout (Figure 3). The hub is built as two units on custom printed circuit boards. A **windscreen pod** behind the rear-view mirror holds the camera, the microcontroller, the IMU and the GNSS receiver. An **under-dash power box** holds everything that switches current: the actuator driver, the relays, the kill relay and the connection to the car’s OBD-II port. A 25-way cable joins them. The LiDAR sits on the roof, tilted forward, and the emergency stop and gesture sensor are within the driver’s reach.

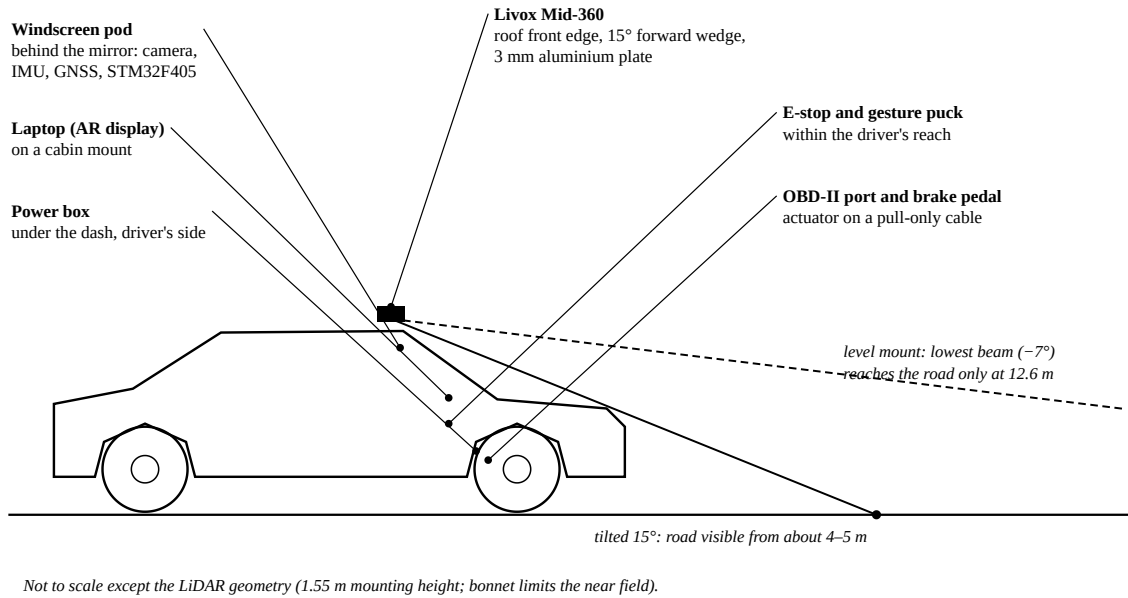


Figure 3: Installation layout, and the geometry of the tilted LiDAR mount

Inside the host, each subsystem runs in its own thread and communicates through lock-free single-producer, single-consumer ring buffers carrying FlatBuffers messages (Figure 10, Appendix D). A consumer always takes the newest message, so a slow stage (for example, the renderer during a heavy frame) never makes another stage fall behind. The **decision arbiter** is the only component permitted to produce an actuation request, which makes the safety-relevant path short and auditable.

3.3 Hardware Design: the Two-Box Hub

Why two boxes. The camera belongs behind the mirror, where the wipers keep its view clean and it sees roughly what the driver sees. Placing the IMU on the *same rigid carrier plate* as the camera fixes the camera–IMU geometry and lets the IMU measure the camera’s own vibration, which the renderer can use to hold overlays steady over bumps. The actuator driver, relays and OBD connection, by contrast, belong under the dash beside the pedal, fuse box and OBD port, where heavy currents stay short and away from the sensitive receivers (Figure 4).

Pod board (4-layer). An **STM32F405RGT6** (Cortex-M4F, 168 MHz) was chosen for its CAN controllers and hand-solderable LQFP64 package. An 8 MHz crystal is mandatory: USB needs a 48 MHz clock within $\pm 0.25\%$, while the internal RC oscillator is only about 1 % accurate. The IMU (BNO085) and GNSS receiver (u-blox NEO-M8N) are mounted as breakout boards on standoffs in the first version. The pod takes 5 V from the power box *or* from the laptop’s USB port through a diode-OR, so it stays running if either cable is pulled, and can report what happened.

Power board (2-layer, 2 oz copper). The rule that makes the split safe is that **every line from the pod enters the power board through a pull-down resistor**. If the microcontroller is in reset, unpowered, or the cable is unplugged, the brake drive, the cable magnet and every relay read “off” in hardware, with no firmware involved. A 74HCT244 buffer converts the 3.3 V signals for the 5 V module inputs (its TTL input threshold of 2.0 V reads 3.3 V reliably). The 12 V input is protected by fuses, reverse-polarity protection and a TVS diode against load-dump transients, feeding a 60 V-rated buck converter.

Actuation power path. The actuator is fed through a fuse and the contacts of a 40 A automotive relay whose *coil* is powered through the normally-closed contact of a mushroom-head emergency stop. Pressing it, a broken wire or a lost supply all remove actuator power in hardware. A BTS7960 H-bridge drives the actuator with 20 kHz PWM, above the audible range, and an ACS712-20A Hall-effect sensor measures its current independently.

OBD-II over CAN. The hub reads vehicle speed and engine rpm directly from the car's CAN bus using the ISO 15765-4 diagnostic protocol [31], replacing a Bluetooth ELM327 adapter. Used vehicles now imported into Kenya must have been first registered in 2019 or later, and CAN has been the norm for OBD-II since about 2008. The transceiver sits in the power box beside the OBD port, so the hub adds only a short unterminated branch to the car's bus; a 2.5 m branch would cause reflections at 500 kbit/s. The firmware never disturbs the bus. It listens silently before transmitting at each candidate bit rate, sends each read-only request only once, and polls at most about 20 times a second.

Fail-safe actuator design. A lead-screw linear actuator is rejected because it cannot be back-driven: it would hold the brake on after a power cut, and typical units are also too strong (up to 900 N) and too slow (5–15 mm/s). The design uses (i) a **pull-only cable** to the pedal, which goes slack if the driver presses further, so the driver's foot always overrides it; (ii) a **holding electromagnet** in the cable, powered through the kill relay, which drops the cable free whenever power is lost, and whose holding force is a **mechanical force ceiling**; and (iii) a **fast, low-force actuator** of about 50–100 mm/s and 100–200 N. It is subject to supervisor approval before purchase.

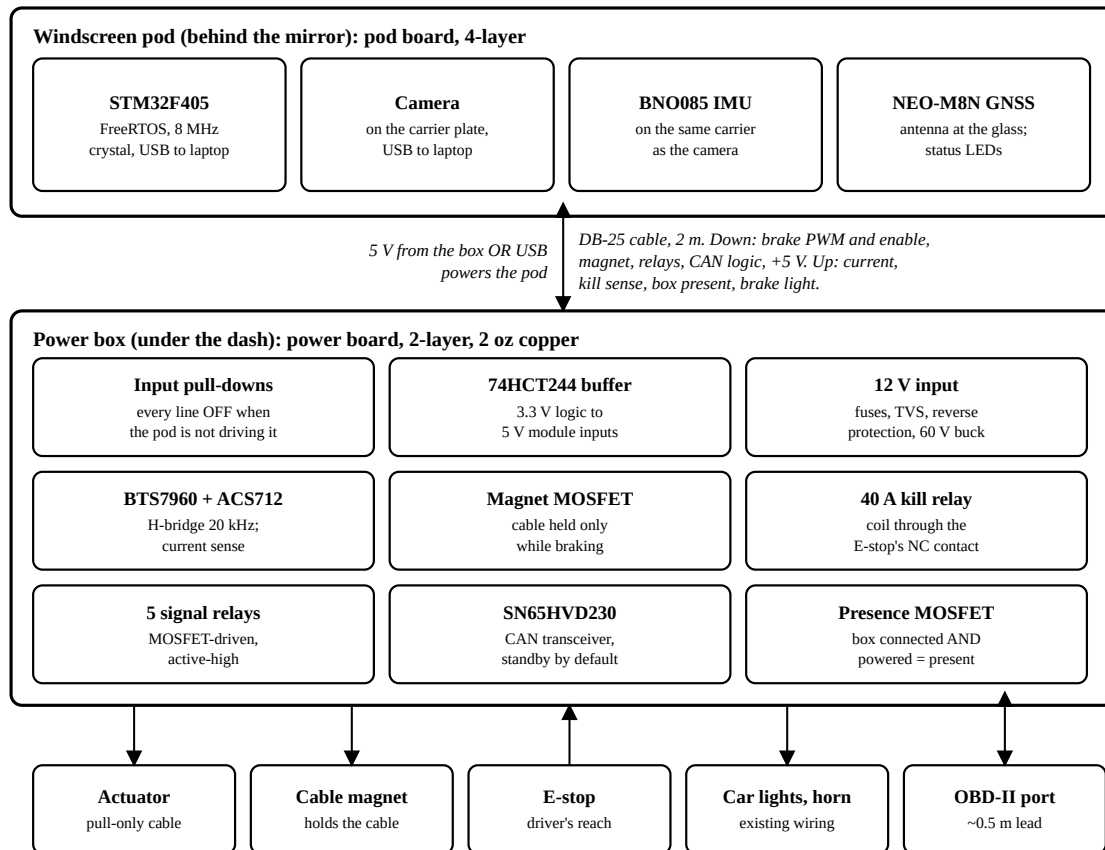


Figure 4: The two-box hub: windscreen pod, inter-box cable and under-dash power box

LiDAR mounting. The manufacturer forbids anything, glass included, in the sensor’s field of view [32]. The unit is also rated only to 55 °C, so it cannot sit behind the windscreen. Its vertical field of view runs from -7° to $+52^\circ$, so mounted level at $h = 1.55$ m its lowest beam meets the road only at

$$d = h / \tan 7^\circ = 1.55 / 0.123 = 12.6 \text{ m} \quad (3.1)$$

Tilted 15° forward on a 3 mm aluminium wedge plate, which also serves as the heatsink the manual asks for, the forward lower edge becomes -22° . The road is then visible from $1.55/\tan 22^\circ \approx 3.8$ m, or from wherever the bonnet stops blocking the view (typically 4–5 m). That covers the 5.0 m gap at which the 10 km/h brake decision is made (Section 3.8). Only the rearward view is lost, and nothing uses it.

Current sensing. The ACS712 outputs 2.5 V at 0 A and 100 mV/A on a 5 V supply. It is divided by 2/3 inside the power box, so that nothing above 3.3 V travels up the cable:

$$V_{ADC} = (2.5 + 0.1 I) \times 2/3 \Rightarrow \text{at the 8 A limit, } V_{ADC} = 2.20 \text{ V} \Rightarrow 2,730 \text{ of } 4,095 \text{ counts} \quad (3.2)$$

One count is 12 mA of actuator current. An unplugged sense line reads about 0 V, which this formula would turn into 25 A, so the firmware ignores the reading whenever the power box is absent. Otherwise, every cable unplug would latch a false overcurrent fault.

Enclosures and power. The pod is printed in ASA or PETG, since PLA softens at about 55 °C and the glass zone gets hotter in the sun. It clips onto a bracket bonded to the glass in the uncoated area near the mirror, and it has a matte hood to stop dashboard reflections reaching the lens. The laptop draws 200–280 W under GPU load, more than a USB-C car charger supplies, so it runs from a 300 W inverter or on its own battery during the short test windows.

3.4 Host–Hub Protocol and the Safety Core

Framing. Every message is framed as `0xAA | type | length (2 bytes, little-endian) | payload | CRC-16/CCITT-FALSE (2 bytes)`. Four message types are used: a 63-byte *SensorReport* from hub to host at 50 Hz; a 9-byte *ActuationCommand* (indicator and light bits, brake intensity 0–255, duration, host timestamp); an 8-byte *AckStatus* reporting what the hub actually applied and any fault; and an empty *Heartbeat* the host must send at least 10 times a second. The payload structures are compiled on both sides with static size checks, so a layout mismatch becomes a build error rather than silently wrong data. The *SensorReport* stream needs $(63 + 6) \times 50 = 3.45$ kB/s, a negligible load for full-speed USB. The parser re-synchronises on the start byte after any corrupt frame; if more than 10 of the last 50 frames are corrupt, actuation is disarmed until 20 clean frames arrive.

Tasks. The firmware has four FreeRTOS tasks: a sensor task that assembles the 50 Hz report; a comms task that parses and sends frames; an actuation task that applies commands every 10 ms; and a watchdog task that checks the link every 20 ms. All safety decisions are made by a single, hardware-independent *SafetyCore* module, which is unit-tested on the laptop.

Safety state machine (Figure 9, Appendix C). The hub boots *disarmed* with the actuator released before any task runs. It arms only while it receives fresh heartbeats, its own report loop is alive, the emergency stop is released, the power box is connected and powered, and the frame-error rate is acceptable. Any failure of these returns it to disarmed, releasing the brake and forgetting the last

command. A brake command is clamped to an intensity of 90 (of 255) and a duration of 1.5 s regardless of what the host asks for. An actuator current above 8 A for 30 ms latches a fault that only a power cycle clears. The watchdog task releases the actuator directly, without waiting on any shared lock, and the IWDG hardware timer resets the whole microcontroller if the firmware itself hangs for 500 ms.

The brake force is therefore limited in four independent places: the host arbiter’s configured intensity (60 of a maximum 90), the hub’s compiled ceiling of 90, the driver’s own clamp in the actuator driver, and the electromagnet’s holding force.

3.5 Perception

Camera pipeline. A 2K (2560×1440) USB camera with manually locked focus supplies 30 frames per second. Autofocus is disabled because every focus change would invalidate the camera calibration. Each frame is timestamped at capture and shared, not copied, with all consumers.

Neural networks. Four networks are exported to ONNX and compiled into TensorRT FP16 engines for the RTX 5060 (Table 2). Pre-processing (letterboxing, normalisation) runs on the GPU in CUDA, and each network’s output is decoded on the host into boxes, masks, lane points or a depth map (Figure 11, Appendix D).

Table 2: Neural networks used by the perception subsystem

Task	Model	Input	Output
Objects and masks	YOLOv8m-seg (COCO)	1280×736	Boxes, classes and a mask per object (vehicle, pedestrian, cyclist, obstacle)
Road signs	YOLOv8, fine-tuned on MTSD	1280×736	29 sign classes, including speed-limit values
Lanes	UFLDV2, ResNet-18 (CULane)	1600×320 band	Up to four lane lines
Depth	MiDaS v2.1 Small	448×256	Dense relative inverse depth

Frame budget. At 30 frames per second each frame has $1000/30 = 33.3$ ms. The four networks were measured running together on recorded 2K road video at a median of 17.8 ms per frame (95th percentile 19.9 ms, maximum 26.2 ms), leaving margin for decoding, fusion and rendering, which run concurrently on other threads.

Road-sign detector for Kenya. No public Kenyan sign dataset exists, so the sign detector is trained on the global MTSD [15], restricted to 29 classes that occur on Kenyan roads, and then audited on openly licensed Kenyan street images. The adopted model reaches mAP50 of 0.716 (mAP50-95 0.581) on a held-out validation set and classified 24 of 25 Kenyan audit signs correctly at a confidence of 0.5 or more (Figure 15, Appendix F). A later variant trained for longer was evaluated and rejected because it did not improve on these figures. A sign tracker confirms a sign over several frames before it is displayed, to avoid flicker.

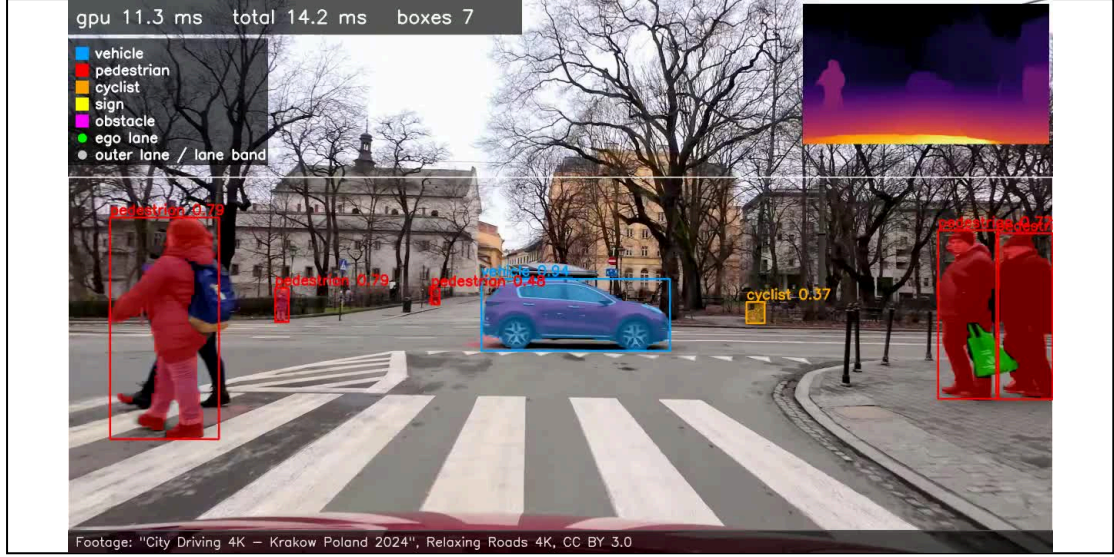


Figure 5: Output of the perception pipeline on recorded city footage: instance masks for pedestrians, a cyclist and vehicles, with the relative depth map inset (footage: “City Driving 4K – Krakow Poland 2024”, Relaxing Roads 4K, CC BY 3.0)

3.6 Sensor Fusion, Tracking and Motion Prediction

Ego-motion (EKF). The car’s own state $x = [p_x, p_y, \psi, v, \omega]$ (position in a local metric frame, heading, speed, yaw rate) is estimated by a 5-state extended Kalman filter [22]. It predicts at the IMU rate with the constant turn rate and velocity (CTRV) model

$$p_x' = p_x + (v/\omega)[\sin(\psi + \omega\Delta t) - \sin\psi], \quad p_y' = p_y + (v/\omega)[\cos\psi - \cos(\psi + \omega\Delta t)], \quad \psi' = \psi + \omega\Delta t \quad (3.3)$$

(with the straight-line limit when $\omega \rightarrow 0$) and corrects with GNSS positions and OBD-II wheel speed using the standard update $K = PH^T(HPH^T + R)^{-1}$, $x \leftarrow x + K(z - h(x))$. GNSS latitude and longitude are converted to the local frame with the WGS-84 meridian and prime-vertical radii of curvature.

Mask-based camera–LiDAR fusion. The Livox Mid-360 produces about 200,000 points per second over $360^\circ \times 59^\circ$. For each camera frame the fusion step (i) moves every LiDAR point to where it would have been seen at the camera’s timestamp, using the ego speed and yaw rate (without this, a 100 ms scan at 10 m/s smears objects by 1 m); (ii) removes points within 0.15 m of the fitted road plane; (iii) projects the rest into the image with the calibrated lens model; (iv) keeps only the nearest point per small image cell, since the LiDAR can see points the camera cannot; (v) assigns points to an object only if they fall inside its *eroded* mask; and (vi) takes the object’s range from the nearest depth cluster holding at least 20% of its points, never the mean, so a few background points cannot move a pedestrian several metres. Objects with fewer than three points receive a MiDaS-based estimate scaled against LiDAR-ranged objects in the same frame, marked *estimated* and never used for braking. LiDAR clusters in the car’s path that no detection explains become *unknown obstacles*: the camera can add information but can never veto what the LiDAR physically measures.

Tracking. Each object is tracked by an IMM filter [23] mixing constant velocity, CTRV and stopping models. Detections are associated with tracks by the Hungarian algorithm [24] within gates derived from each track’s innovation covariance [25]. A track is confirmed after repeated detections and new

tracks must pass a per-class maximum-speed check, which prevents false associations from creating implausible tracks.

Motion prediction. For every track the predictor produces a position distribution up to 2 s ahead (drawn in the AR view as a ribbon whose width shows uncertainty), the closest point of approach to the ego car, and a Monte-Carlo collision probability. With relative position $\mathbf{r}$ and relative velocity $\mathbf{v}$:

$$t^* = -(\mathbf{r} \cdot \mathbf{v})/|\mathbf{v}|^2, \quad d^* = |\mathbf{r} + \mathbf{v}t^*| \quad (3.4)$$

Unlike time to collision, the closest point of approach also captures crossing pedestrians and vehicles cutting in. Predictions feed warnings only; they never trigger the brake (Section 3.8).

3.7 Precision Navigation

The navigation pipeline answers three questions: which road to take, where the car is on it, and where that road is in the camera image (Figure 12, Appendix D).

Routing. OSRM [26] runs locally with the multi-level Dijkstra algorithm on an OpenStreetMap extract covering everything within 100 km of Nairobi’s CBD (as far as Thika, Machakos, Naivasha and beyond). A script downloads the daily Kenya extract, verifies its checksum, clips it, builds the routing data, and only switches to the new map after a smoke-test route from the CBD to Thika succeeds; any failure leaves the previous map in service. The Nairobi CBD to Thika route is 43.7 km. MLD was chosen because it allows road weights to be changed in seconds [27]: when a closure is detected, the route is recomputed in about 1.2 s.

Road status in three layers. The map is treated as an expectation, and the sensors as the truth. Layer 1 is the regularly refreshed OpenStreetMap base. Layer 2, a live traffic and incident feed from a commercial provider, is designed but deferred until its licence terms permit this use. Layer 3 is the car’s own observations (a closed road, a blocked lane), which always override the map.

Map matching. The fused pose, not raw GNSS, is matched to the road network with the HMM method of Newson and Krumm [28], using a sliding trace of recent positions. The matcher reports which road the car is on and its distance along the route. It is throttled to about 10 Hz and resets its trace after a gap of more than 10 s. On a recorded 2,773-fix drive in Nairobi, the median distance from fix to matched road was 3.1 m and 98.7% of fixes were within 20 m; the remaining fixes were a genuine deviation from the planned route, which the matcher correctly reported as off-route.

Road-surface projector. This is the step that makes the route appear painted on the road. From the matched position it samples the route every 1.5 m out to 60 m ahead and gives each point a height: the mean of measured LiDAR ground points within 1 m where available; the fitted road plane where the road is hidden (for example by a car ahead); and flat ground beyond LiDAR range. Two corrections come from the detected lane lines. A *heading* correction fixes compass error, which matters because a 3° heading error displaces a point 60 m ahead by $60 \times \tan 3^\circ = 3.1$ m, a whole lane. A *lateral* correction moves the line from the road’s centre line (which is what OpenStreetMap records) into the car’s lane. Both are persistent estimates that a single frame can change only by a bounded step, so one bad lane detection cannot throw the line across the road; frames are used only when both lane lines are found and the lane is 2.3–5 m wide.

3.8 Hazard Assessment and the Decision Arbiter

Hazard assessment. A rule-based classifier flags, for each confirmed track: tailgating (the ego car's time gap to the vehicle ahead below 1.0 s); erratic speed (speed variation left after removing a steady trend); swerving (lateral wander after removing a steady turn); sudden braking (a high IMM stopping-model probability); crossing (a closest-approach conflict with high collision probability); forward collision risk; and a vulnerable road user in the path. These combine into a risk value $r \in [0, 1]$ that sets the colour and motion of the hazard overlay. Rules were chosen over a learned classifier because each flag can be explained and tested.

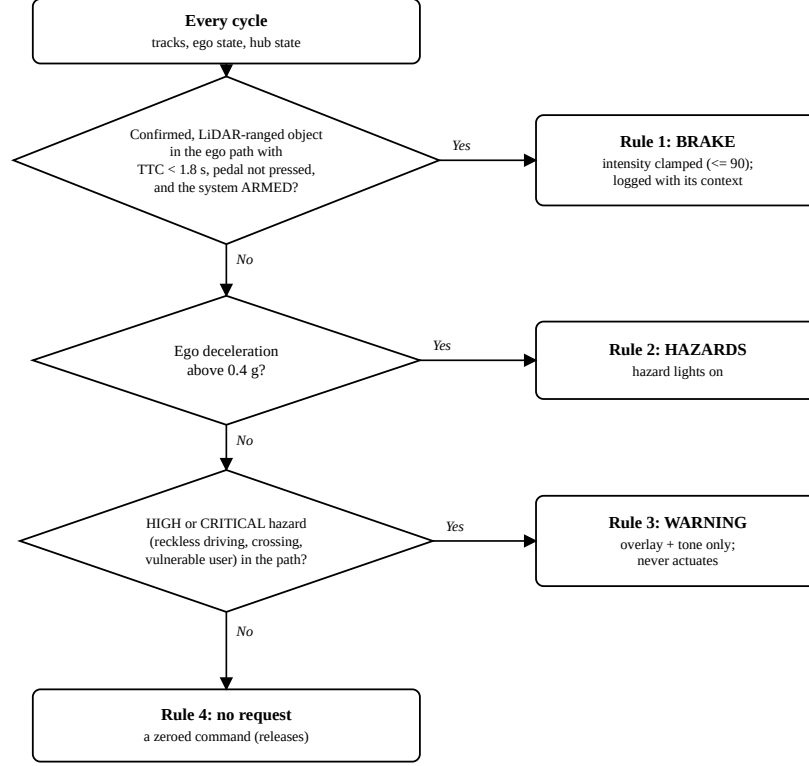
Decision arbiter (Figure 6). Four rules are evaluated in priority order every cycle, and the first that matches wins:

1. **Brake** when the time to collision with a confirmed object in the ego path, whose range was measured by the LiDAR within the last 200 ms, is below 1.8 s, the driver is not already braking, and the system is armed (fresh hub report, emergency stop released, no actuator fault, and the evidence log able to record);
2. **Hazard lights** when the car itself decelerates harder than 0.4 g;
3. **Warning** (overlay and tone only) for a high or critical hazard in the path;
4. otherwise, a zeroed request that releases everything.

Only rule 1 can brake, and it uses only the *current measured* gap and closing speed:

$$TTC = gap / closing\ speed, \quad closing\ speed > 0.5\ m/s \quad (3.5)$$

Braking on predictions or probabilities would risk phantom braking, which is itself a hazard. Every request, and the release that follows it, is written to the evidence log with its full context before it is sent.



Predictions, collision probabilities, signs and map data never reach rule 1; they drive warnings only.

Figure 6: Decision arbiter rules, in priority order

Is 1.8 s enough at test speeds? At the 10 km/h upper limit of the campus tests ($v = 2.78$ m/s), braking starts when the gap is $2.78 \times 1.8 = 5.0$ m. The expected latency from capture to pedal movement is about 0.6 s (one frame of 33 ms, about 20 ms of inference, fusion and arbitration, under 20 ms of USB and hub cycle time, and up to 0.4 s for a 50–100 mm/s actuator to take up about 40 mm of pedal travel), during which the car covers $2.78 \times 0.6 = 1.7$ m. At a moderate deceleration of 0.3 g (2.94 m/s²) the stopping distance is

$$s = v^2/2a = 2.78^2/(2 \times 2.94) = 1.3 \text{ m} \quad (3.6)$$

so the car stops within about 3.0 m of the trigger point, leaving about 2 m of margin. The threshold is a conservative starting value to be tuned during testing, not a final figure; its behaviour was verified in simulation, where an approaching object first triggered braking at a true TTC of 1.70 s, inside the 1.2–1.9 s acceptance window.

3.9 AR Rendering and Display

The renderer composes the camera image and all overlays on the GPU with OpenGL 4.6. Under WSL2 this runs through Mesa’s Direct3D 12 translation on the NVIDIA GPU; the renderer checks at start-up which GPU it received and refuses a software fallback. Each frame, the video is uploaded as a texture and the overlays are drawn in one pass from a compact *overlay scene* produced by the other subsystems. The main overlay types are:

- the **route band**, drawn in 3D on the projected road surface with perspective-correct interpolation, solid where the surface is measured and fainter in the far field;

- **hazard glows** following each object’s mask, coloured and animated by risk level, with predicted-path ribbons, a following-distance zone for tailgating, and a “barrier” for a stop sign;
- **sign badges** (for example, speed-limit grading) and turn instructions.

Overlays never hide a hazard: route graphics are occluded by objects in front of them using each object’s measured depth. The 3D overlays are moved by the car’s own motion between the moment their pose was computed and the moment the frame is shown, which compensates for processing latency. The display is behind a *DisplaySink* interface; the only sink built now is a window on the laptop screen, and an optical projector sink can be added later without changing the rest of the system.

Measured at 2560×1440 on real road video with a realistic overlay load, the renderer ran at 54–82 frames per second (target 24 or more), with the overlay pass taking about 2 ms and the video upload 7–10 ms; the shader’s projection agreed with the calibrated camera model to within 2 pixels (Figure 7).



Figure 7: AR renderer output: route band, hazard glows and overlays composited on road video at 2560×1440 (synthetic overlay scene, estimated camera model; footage: Relaxing Roads 4K, CC BY 3.0)

3.10 Calibration

Camera intrinsics (focal lengths, principal point and the five distortion coefficients k_1, k_2, p_1, p_2, k_3) are estimated with Zhang’s checkerboard method [17], with focus locked. **Extrinsics** relate the camera, the LiDAR and the vehicle frame (x forward, y left, z up, origin on the ground below the rear axle). The LiDAR–camera transform is estimated from a calibration target and then *checked at every start-up* by measuring how well LiDAR points fall inside detected object masks; a falling score over time signals that a mount has moved. A repeatable **road-projection check**, driving slowly along a marked road and confirming the route stays in the lane without floating or sinking, follows every recalibration.

IMU mounting. The IMU is tilted with the camera, so its readings are rotated into the vehicle frame before fusion. Read raw at a 12° pitch, the measured yaw rate would be 2 % low and would pick up a fifth of the body’s roll rate in corners. The rotation is calibrated from data the hub already reports. Roll and pitch come from gravity, averaged while the car is at rest on level ground ($\text{roll} = \text{atan2}(a_y,$

a_z), $\text{pitch} = \text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$). Yaw comes from straight-line acceleration and braking, with the sign taken from OBD speed. At every stop, the angle between measured and expected gravity reveals a knocked pod.

3.11 Testing and Staged Validation

Software tests. Every module has unit tests (GoogleTest on the host; Unity for the hub’s protocol and safety logic, which is compiled to run natively on the laptop), run with address and undefined-behaviour sanitizers. Safety-critical code is also tested by *fuzzing*: 200,000 random inputs to the decision arbiter must never produce a brake above the ceiling, and 600,000 random events, including cable unplugs, must never leave the hub’s actuator applied in an unsafe state. Tests are themselves checked by *mutation*: deliberately breaking a safety rule must make a test fail. Integration tests replay recorded video and drives.

Actuator bench gate. Before any in-vehicle work, the actuator is mounted with a real used brake pedal on a bench rig with a load cell, and must pass six checks on three consecutive attempts: (1) the rig check, confirming the actuator and pedal are on the bench and not in a vehicle; (2) a sweep across brake intensities, logging the intensity the hub actually applied (evidence that its ceiling holds whatever is asked), peak current, peak force and response time; (3) **a person pushes the pedal back by foot while the actuator holds its maximum intensity**, the most important check in the project; (4) the emergency stop is pressed mid-actuation and the drive and current must fall to zero; and (5) the USB cable is physically unplugged mid-actuation and the actuator must release, and its current fall to zero, within 200 ms with no software command. The hub measures (5) itself and reports it once the cable is reconnected. In check (6), the inter-box cable is pulled mid-actuation: the actuator must go limp at once, and the hub must report the fault without latching a false overcurrent. Only after three consecutive passes is the maximum brake intensity treated as final. The whole checklist is then repeated on the custom PCB hardware, since new hardware needs its own evidence.

Staged validation then proceeds in four stages, each with a written checklist (Appendix E): **A**, the bench gate above; **B**, stationary in the vehicle, with the handbrake on and wheels chocked; **C**, an empty field with a spotter, at low speed; and **D**, a formally closed campus road section at 5–10 km/h, with spotters, an agreed abort signal, and a manual confirm-to-brake step on the first runs before automatic braking is allowed.

3.12 Demonstration and Presentation Plan

The presentation will follow the same staged-risk order as the testing: it opens with video of the bench test, showing a foot overriding the actuator and the watchdog release, so the safety story is established before anything else. It then shows the AR display live on the stationary car, with a person walking into view to trigger detection, highlighting and (where the venue allows) a visible, overridable actuation. Moving tests are shown as recorded video rather than attempted live. Results are presented as measured figures (frame rate, inference time, TTC accuracy, actuator response time, navigation lateral error) drawn directly from the evidence log, and the talk closes with the future optical display, which the DisplaySink interface is designed to accept.

4.0 Expected Results

The expected results are set out per specific objective. Because the software was developed ahead of the hardware, several results have already been measured on recorded data and in simulation; these are reported as *achieved to date*, and the remaining results will be measured on the hardware and in the vehicle.

4.1 Objective 1: Perception and Sensor Fusion

A perception pipeline that processes every frame of 2K video in real time on the laptop GPU, producing for each frame: instance masks and classes for vehicles, pedestrians, cyclists and obstacles; Kenyan road signs; lane lines; and a relative depth map (Figure 13 and Figure 14, Appendix F). With the LiDAR installed, each detected object will carry a measured range, and tracks will carry velocity, predicted path and collision probability.

4.2 Objective 2: Road-Locked AR Navigation

An AR display in which the route appears painted on the road in the car's lane, following slopes and dips within LiDAR range and fading honestly to a flat-road estimate beyond it; offline turn-by-turn routing anywhere within 100 km of Nairobi, with fast rerouting around closures; and sign, hazard and turn overlays that never hide a real hazard. The road-projection check (Section 3.10) is expected to show the route line inside the correct lane within LiDAR range.

4.3 Objective 3: Hazard Assessment and Fail-Safe Actuation

Hazard and reckless-driving warnings shown as colour-graded glows and zones; automatic hazard lights under hard braking; and a brake actuation that engages only at a measured TTC below 1.8 s, is always overridable by the driver's foot, is cut instantly by the emergency stop, and releases within 200 ms of losing the laptop, with every event recorded in the evidence log. The bench rig will produce force, current and response-time curves across intensities, and video of the override, kill-switch and unplug tests.

4.4 Summary of Targets and Results to Date

Table 3: Measurable targets per objective, with results achieved to date

Obj.	Metric	Target	Achieved to date	Method
1	Four networks together, per 2K frame	≤ 33 ms	Median 17.8 ms; p95 19.9 ms; max 26.2 ms	Recorded road video, RTX 5060
1	Road-sign detection accuracy	$\text{mAP}_{50} \geq 0.70$	0.716 (mAP_{50-95} 0.581); Kenyan audit 24/25	Held-out MTSD set; Kenyan images
1	LiDAR range per detected object; mask-point purity	Range for every object with ≥ 3 points	Verified on synthetic scenes	In-vehicle, Stage B (pending hardware)
2	AR render rate at 2560×1440	≥ 24 fps	54–82 fps; overlay pass ≈ 2 ms	Render benchmark
2	Overlay projection agreement with camera	≤ 2 px	Within 2 px	Automated projection test

model				
2	Map-matching accuracy	Correct road; median < 5 m	Median 3.1 m; 98.7% < 20 m (2,773 fixes)	Recorded Nairobi drive
2	Reroute after a closure	< 2 s	1.2 s	OSRM MLD re-weighting
2	Route line in the correct lane, near field	Inside the lane within LiDAR range	Pending hardware	Road-projection check
3	Brake trigger time	TTC 1.2–1.9 s (threshold 1.8 s)	First brake at true TTC 1.70 s	End-to-end simulation
3	Brake never above the ceiling	0 violations	0 in 200,000 random inputs (host); 0 in 600,000 events (hub)	Fuzz tests
3	Watchdog release after link loss	≤ 200 ms	Logic verified in unit tests	Bench, USB unplugged (pending)
3	Driver override at maximum intensity	Pedal moves by foot, 3/3 attempts	Pending hardware	Bench rig, load cell, video
3	Power box disconnected mid-brake	Released at once; fault reported; no false latch	Logic verified in unit tests and fuzz	Bench, check 6 (pending)
3	OBD-II over the car's CAN bus	Never transmits at a wrong bit rate	Verified against a simulated car	Stationary car, Stage B
All	Automated tests passing	100%	Host 232/232; routing and GPU 29/29; firmware 38/38	Test suites with sanitizers

5.0 Project Time Plan

Figure 8 shows the project schedule. The software-only phases are complete. The critical path now runs through procurement: the LiDAR, the BNO085 IMU and the actuator are not stocked locally and will be imported in a single consignment, so their lead time (including customs clearance) governs when LiDAR integration and the bench gate can start. Work that does not depend on the imports (hub bring-up with local parts, camera calibration, the bench rig itself) proceeds in parallel. The custom PCBs (Phase 12B) are designed while the bench gate runs on module hardware, and must then pass the same gate again.

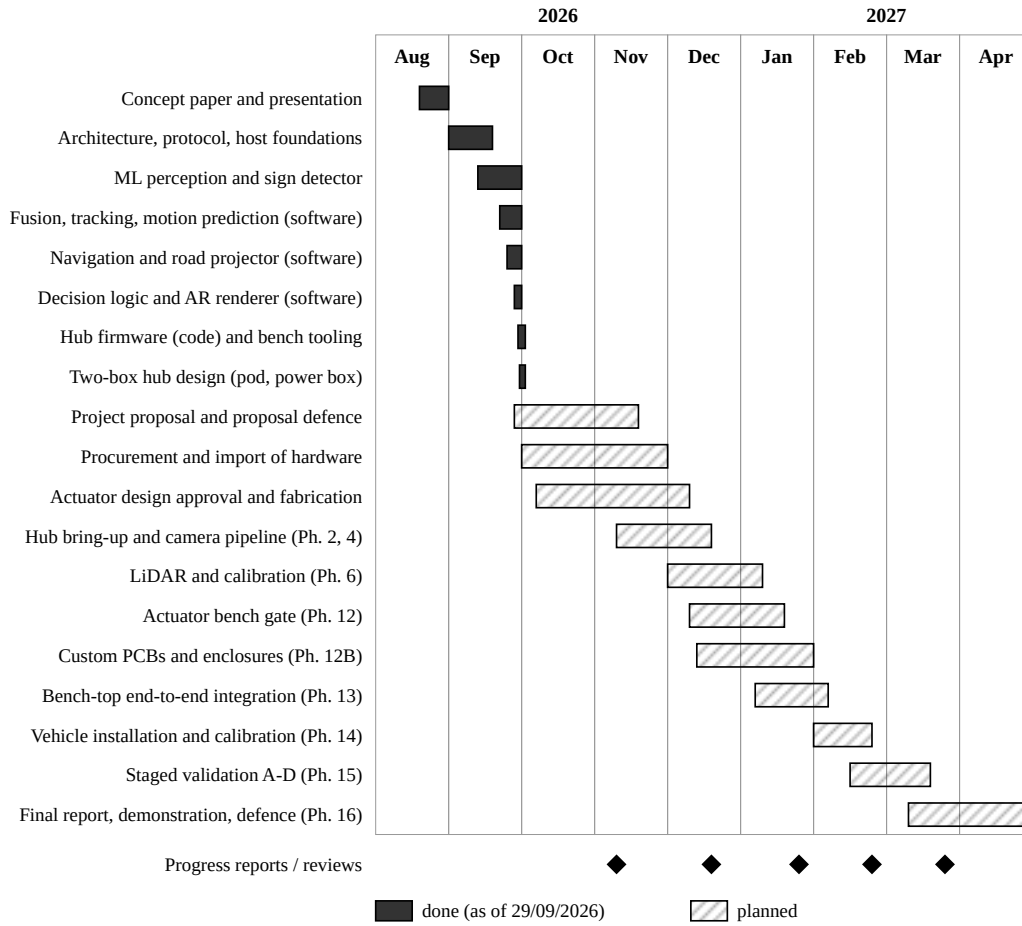


Figure 8: Project time plan (Gantt chart), August 2026 to April 2027

Table 4: Development phases, their exit criteria and status

Phase	Work	Exit criterion (summary)	Status
0–1	Scaffold and environment; shared protocol	Toolchain verified; protocol round-trips on both sides	Done
2	Hub bring-up	Sensors read; relays switch; watchdog releases on the bench	Firmware written; needs hardware
3	Host skeleton and hub link: message bus, event log, vehicle interface	Host talks to the real hub firmware	Done (software)
4	Camera pipeline	2K capture at 30 fps with locked focus	Needs camera
5	ML inference (four networks)	All four within the frame budget	Done
6	LiDAR and calibration	Point cloud aligns with camera detections; ground model published	Needs LiDAR
7	Fusion, tracking, prediction	Tests on synthetic and recorded data	Done (software)
8	Navigation: routing, map matching, road status	Offline routing; recorded-drive matching	Done
9	Road-surface projector; sign detector evaluation	Projection tests; sign model adopted	Done (software)

10	Hazard classifier and decision arbiter	Rule tests; end-to-end TTC; fuzz	Done
11	GPU AR renderer and display	≥ 24 fps; projection ≤ 2 px	Done
12	Actuator bench gate	Six checks, three consecutive passes	Firmware and tools ready; needs actuator
12B	Custom hub PCBs (pod, power box) and enclosures	Six checks passed again on the PCB hardware	Designed; needs fabrication
13	Bench-top end-to-end integration	Full system on the bench, no vehicle	Planned
14	Vehicle installation and calibration	Mounted, calibrated, Stage B checklist	Planned
15	Staged real-world validation	Checklists complete; evidence logged	Planned
16	Final report, demonstration, defence	Report submitted; demonstration given	Planned

6.0 Budget Estimates

Prices were researched from Nairobi suppliers in September 2026. Items not sold in Kenya are grouped into one import consignment, since duty and clearing are charged per consignment. The laptop is already owned. Table 5 summarises the budget; the itemised list is in Appendix A.

Table 5: Budget summary (Kenya shillings)

Block	Main items	Estimate (KES)
Hub and locally bought sensors	STM32F405 board, ST-Link, DB-25 cable, CAN transceiver, OBD lead, GNSS, gesture sensor, camera	15,000 – 26,000
Actuation and safety (excluding the actuator)	BTS7960, ACS712, E-stop, kill relay, five automotive relays, optocoupler	5,500 – 9,500
Power (excluding laptop power)	60 V-rated buck converter, TVS and reverse protection, fuses, automotive wire	3,000 – 4,000
Bench rig	12 V supply, load cell and HX711, used brake pedal, base	8,000 – 13,000
Installation	Pod bracket, LiDAR wedge plate, filament, power-box enclosure, laptop mount	10,000 – 18,000
Local subtotal		41,000 – 71,000
Import consignment	Mid-360 LiDAR and cable, BNO085, M12 camera; landed cost with duty and VAT	140,000 – 170,000
Custom PCBs	Pod board (4-layer) and power board (2-layer), components, courier and duty	15,000 – 28,000
Actuator and laptop in-car power	Actuator and holding magnet (pending approval); inverter if needed	To be confirmed
Total		$\approx 195,000 - 270,000$

The LiDAR is about two-thirds of the total and cannot be dropped without losing the measured road surface and the metric range used for braking. A clearing agent's quote will be obtained before ordering.

References

- [1] World Health Organization, *Global Status Report on Road Safety 2023*. Geneva, Switzerland: WHO, 2023.
- [2] National Transport and Safety Authority (NTSA), “Road traffic crash statistics, 2022–2023,” Nairobi, Kenya, 2023, as reported in *The Star* (Kenya), Nov. 2023.
- [3] S. G. Klauer, T. A. Dingus, V. L. Neale, J. D. Sudweeks, and D. J. Ramsey, “The impact of driver inattention on near-crash/crash risk: An analysis using the 100-car naturalistic driving study data,” Nat. Highway Traffic Safety Admin., Washington, DC, USA, Tech. Rep. DOT HS 810 594, 2006.
- [4] T. A. Dingus, F. Guo, S. Lee, J. F. Antin, M. Perez, M. Buchanan-King, and J. Hankey, “Driver crash risk factors and prevalence evaluation using naturalistic driving data,” *Proc. Nat. Acad. Sci. USA*, vol. 113, no. 10, pp. 2636–2641, 2016.
- [5] J. L. Gabbard, G. M. Fitch, and H. Kim, “Behind the glass: Driver challenges and opportunities for AR automotive applications,” *Proc. IEEE*, vol. 102, no. 2, pp. 124–136, Feb. 2014.
- [6] K. Bark, C. Tran, K. Fujimura, and V. Ng-Thow-Hing, “Personal Navi: Benefits of an augmented reality navigational aid using a see-thru 3D volumetric HUD,” in *Proc. 6th Int. Conf. Automotive User Interfaces and Interactive Vehicular Applications (AutomotiveUI ’14)*, Seattle, WA, USA, 2014, pp. 1–8, doi: 10.1145/2667317.2667329.
- [7] J. B. Cicchino, “Effectiveness of forward collision warning and autonomous emergency braking systems in reducing front-to-rear crash rates,” *Accid. Anal. Prev.*, vol. 99, pp. 142–152, Feb. 2017.
- [8] H. Kim and J. L. Gabbard, “Assessing distraction potential of augmented reality head-up displays for vehicle drivers,” *Human Factors*, vol. 64, no. 5, pp. 852–865, 2022, doi: 10.1177/0018720819844845.
- [9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.
- [10] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, “A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Mach. Learn. Knowl. Extr.*, vol. 5, no. 4, pp. 1680–1716, 2023.
- [11] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, “YOLACT: Real-time instance segmentation,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, Seoul, South Korea, 2019, pp. 9157–9166.
- [12] Z. Qin, H. Wang, and X. Li, “Ultra fast structure-aware deep lane detection,” in *Proc. European Conf. Computer Vision (ECCV)*, Glasgow, U.K., 2020, pp. 276–291.
- [13] Z. Qin, P. Zhang, and X. Li, “Ultra fast deep lane detection with hybrid anchor driven ordinal classification,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 5, pp. 2555–2568, May 2024.
- [14] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, Mar. 2022.
- [15] C. Ertler, J. Mislej, T. Ollmann, L. Porzi, G. Neuhold, and Y. Kuang, “The Mapillary traffic sign dataset for detection and classification on a global scale,” in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 68–84.
- [16] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [17] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 11, pp. 1330–1334, Nov. 2000.

- [18] C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas, “Frustum PointNets for 3D object detection from RGB-D data,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 918–927.
- [19] S. Vora, A. H. Lang, B. Helou, and O. Beijbom, “PointPainting: Sequential fusion for 3D object detection,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 4604–4612.
- [20] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Trans. ASME, J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.
- [21] S. J. Julier and J. K. Uhlmann, “Unscented filtering and nonlinear estimation,” *Proc. IEEE*, vol. 92, no. 3, pp. 401–422, Mar. 2004.
- [22] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [23] H. A. P. Blom and Y. Bar-Shalom, “The interacting multiple model algorithm for systems with Markovian switching coefficients,” *IEEE Trans. Autom. Control*, vol. 33, no. 8, pp. 780–783, Aug. 1988.
- [24] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Res. Logist. Quart.*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [25] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York, NY, USA: Wiley, 2001.
- [26] D. Luxen and C. Vetter, “Real-time routing with OpenStreetMap data,” in *Proc. 19th ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems*, Chicago, IL, USA, 2011, pp. 513–516.
- [27] D. Delling, A. V. Goldberg, T. Pajor, and R. F. Werneck, “Customizable route planning,” in *Experimental Algorithms (SEA 2011)*, Lecture Notes in Computer Science, vol. 6630. Berlin, Germany: Springer, 2011, pp. 376–387.
- [28] P. Newson and J. Krumm, “Hidden Markov map matching through noise and sparseness,” in *Proc. 17th ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems*, Seattle, WA, USA, 2009, pp. 336–343.
- [29] D. N. Lee, “A theory of visual control of braking based on information about time-to-collision,” *Perception*, vol. 5, no. 4, pp. 437–459, 1976.
- [30] *Road Vehicles — Functional Safety*, ISO 26262:2018, International Organization for Standardization, Geneva, Switzerland, 2018.
- [31] *Road Vehicles — Diagnostic Communication over Controller Area Network (DoCAN) — Part 4: Requirements for Emissions-Related Systems*, ISO 15765-4:2021, International Organization for Standardization, Geneva, Switzerland, 2021.
- [32] Livox Technology, *Livox Mid-360 User Manual*, v1.2. Shenzhen, China: Livox, Apr. 2024.

Appendix A: Component Specifications

Table 6: Itemised components, key specifications and sources

Item	Specification that matters	Source	KES
Sensor and actuator hub			
STM32F405RGT6 development board	Bring-up before the pod board; two CAN controllers; 8 MHz crystal	K-Technics	3,200
ST-Link V2	SWD flashing and debugging	K-Technics	750
DB-25 connectors, shielded 25-core cable (2 m)	The inter-box cable; every conductor wired	Luthuli Ave / Jumia	1,000–1,500
74HCT244, MOSFETs, resistors, diodes	Buffered outputs with fail-safe pull-downs	Pixel / Nerokas	500–1,000
SN65HVD230 CAN transceiver; OBD-II lead	OBD over the car’s CAN bus, short stub	Pixel / Jumia	1,200–2,200
Perfboard, headers, terminals	Bring-up versions of both boxes	K-Technics / Pixel	≈ 1,000
Sensors			
2K USB camera	2560×1440, 30 fps, manual focus lock over UVC (bench)	Jumia	6,000–13,000
2K UVC board camera, M12 lens	For the pod: locked focus, glass lens, ≥ 60 °C rating	Import	see §6
Livox Mid-360 LiDAR + cable	360°×59°, ≈ 200,000 pts/s, Ethernet, 9–27 V	Import	see §6
u-blox NEO-M8N GNSS	Multi-constellation, UART, external antenna	Pixel / Nerokas	1,800
BNO085 IMU	On-chip sensor fusion, I2C	Import	see §6
ELM327 + HC-05 (fallback only)	Only if the test car predates CAN-based OBD-II	Jumia / K-Technics	(1,600–2,650)
APDS-9960 gesture sensor	I2C; driver-hands-free menu control	Pixel	800
Actuation and safety			
BTS7960 H-bridge	43 A, PWM, enable inputs	Jumia	1,200–1,800
ACS712-20A	Hall-effect, 100 mV/A, independent current reading	Nerokas / K-Technics	400–600
Emergency-stop switch	22 mm mushroom, twist release, NC + second contact block	Luthuli Ave / Industrial Area	800–1,500
40 A automotive relays (×2)	E-stop switches the coil; contacts carry actuator current	Auto-spares	600–1,000
12 V automotive relays (×5)	Indicators, hazards, horn, beam; MOSFET-driven, off by default	Auto-spares	1,500–2,500
PC817 optocoupler	Reads the brake-light switch (driver braking)	Pixel / K-Technics	150–300

Brake actuator + holding magnet	50–100 mm/s, 100–200 N, pull-only cable	Import (pending approval)	TBC
Power, bench rig and installation			
Buck converter 12 V → 5 V; TVS; reverse protection	60 V-rated input survives load-dump transients	Jumia / Nerokas / Pixel	1,300–2,000
Fuses, holders, automotive wire	Separate fused rails: peripherals, actuator, LiDAR	Auto-spares	≈ 2,000
Bench 12 V supply ≥ 10 A	Off-vehicle power for actuator, LiDAR, hub	Jumia	2,500–5,000
50 kg load cell + HX711	Measures applied pedal force	Nerokas	≈ 2,000
Used brake-pedal assembly, rig base	Real pedal, pivot and return spring	Used-parts yards	2,500–4,500
Pod bracket and VHB; 3 mm aluminium 15° LiDAR plate	Repeatable pod position; LiDAR heatsink and tilt	Fabricator	2,500–5,000
ASA/PETG filament, power-box enclosure, glands	Heat-resistant enclosures (not PLA)	Nairobi 3D-print shops	3,500–6,500
Laptop mount, fuse tap, looms	Cabin installation	Jumia / auto-spares	4,000–6,500
Custom PCBs			
Pod board (4-layer) and power board (2-layer, 2 oz), ×5 each	Fabrication, components, courier and duty	JLCPCB / PCBWay	15,000–28,000

Table 7: Landed cost of the import consignment (approx. US\$810 CIF at 129 KES/US\$)

Charge	Basis	Approx.
Import duty	0–25% of CIF depending on classification (worst case assumed)	≤ US\$200
IDF	2.5% of CIF	US\$20
RDL	2% of CIF	US\$16
VAT	16% of (CIF + duty + IDF + RDL)	≈ US\$168
Landed		US\$1,050–1,220 ≈ KES 135k–157k

Appendix B: Host–Hub Protocol Reference

Frame: 0xAA | type (1) | length (2, LE) | payload (length) | CRC (2, LE). The CRC is CRC-16/CCITT-FALSE (polynomial 0x1021, initial value 0xFFFF, no reflection, no final XOR; check value 0x29B1 for “123456789”), computed over type, length and payload. All multi-byte fields are little-endian and packed; payload sizes are asserted at compile time on both the hub and the host.

Table 8: Message types

Type	Name	Direction	Rate	Payload	Contents
0x01	SensorReport	hub → host	50 Hz	63 B	Time; GNSS latitude, longitude, speed, fix; IMU acceleration, rates, heading; OBD speed, rpm, brake pedal; gesture; ignition; kill switch
0x02	ActuationCommand	host → hub	on change	9 B	Indicator bits; light and horn bits; brake request 0–255; duration (ms); host timestamp
0x03	AckStatus	hub → host	per command	8 B	Time; command accepted; fault code; brake intensity actually applied
0x04	Heartbeat	host → hub	≥ 10 Hz	0 B	Liveness only
0x05	BenchTelemetry	hub → host	50 Hz, bench build only	22 B	Current, applied intensity, magnet, armed, fault, load-cell force, kill switch, measured release times after link loss

Watchdog contract. The hub treats the link as lost when no valid frame has arrived for 200 ms. A command that requests an action is accepted only while the hub is armed and only if its host timestamp is newer than the last accepted one, so a stale or replayed frame never acts; a command requesting nothing is always accepted, since it can only release. A brake always ends after its own duration or 1.5 s, whichever is shorter; heartbeats do not extend it.

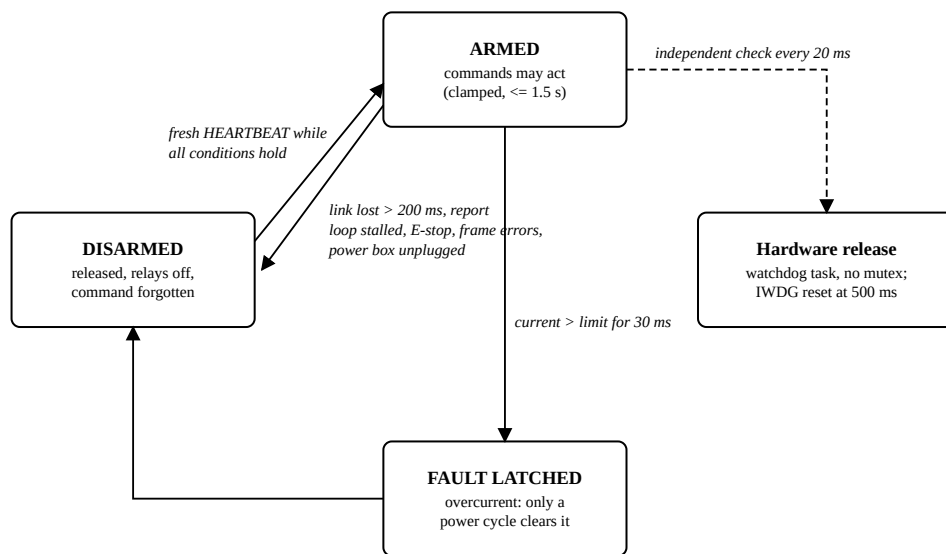
Appendix C: Safety Properties and Fault Codes

The finished system must demonstrate five non-negotiable properties, each verified explicitly:

1. The driver's foot always overrides the actuator (pull-only cable; bench check 3).
2. A physical kill switch removes actuator power independent of software (E-stop in the relay coil loop; bench check 4).
3. The hub releases the actuator within 200 ms of host link loss, independent of host software (watchdog task, IWDG backstop; bench check 5). Across the two-box split, the power board releases in hardware whenever the pod is not driving it (bench check 6).
4. The host never drives actuation hardware directly; it only sends a bounded request, and the hub enforces the real ceiling.
5. Every actuation event (request, hub decision, applied intensity and duration, release reason) is logged with a timestamp on both host and hub.

Table 9: Hub fault codes (*AckStatus.actuatorFaultCode*)

Code	Meaning	Hub behaviour
0	No fault	Normal operation
1	Overcurrent (> 8 A for 30 ms)	Actuator released, relays off; latched until power cycle
2	Kill switch engaged	All actuation disarmed; host informed
3	Host link timeout (> 200 ms)	Watchdog release
4	Frame error rate exceeded (> 10 of 50)	Disarmed until 20 clean frames
5	Power box absent (cable unplugged or box unpowered)	Released (also in hardware); current reading ignored; re-arms on a fresh heartbeat



At boot the hub is **DISARMED** with the actuator released before any task runs.

Figure 9: Hub safety state machine

Hazard analysis (summary). Table 10 lists the main hazards of the actuation function and the independent measures that address each.

Table 10: Actuation hazards and mitigations

Hazard	Mitigations (independent layers)
Brake stuck on after a fault or power loss	Magnet drops the cable when unpowered; no lead screw; hub releases on link loss; IWDG reset
Excessive brake force	Host clamp (60 of 90); hub ceiling 90; overcurrent latch; magnet holding force
Phantom braking	Brake only on measured TTC of a confirmed, LiDAR-ranged object; predictions and ML probabilities excluded; low test speeds; manual confirm-to-brake in first Stage D runs
Driver unable to override	Pull-only cable goes slack; bench check 3 is the project's hard gate
Laptop crash or stall	Hub watchdog (200 ms); commands have a maximum duration; hub boots disarmed
Corrupt or replayed commands	CRC-16 on every frame; stale timestamps rejected; frame-error disarm
Inter-box cable unplugged or cut	Pull-downs on the power board release everything; presence and kill-sense lines both report it (fault 5); floating current reading ignored
Hub disturbs the car's CAN bus	Transceiver in standby by default; silent listening before any transmission; single-shot, read-only requests; short stub
No evidence of what happened	Arbiter refuses to arm if the event log cannot record; hub reports every applied value

Appendix D: Key Algorithms

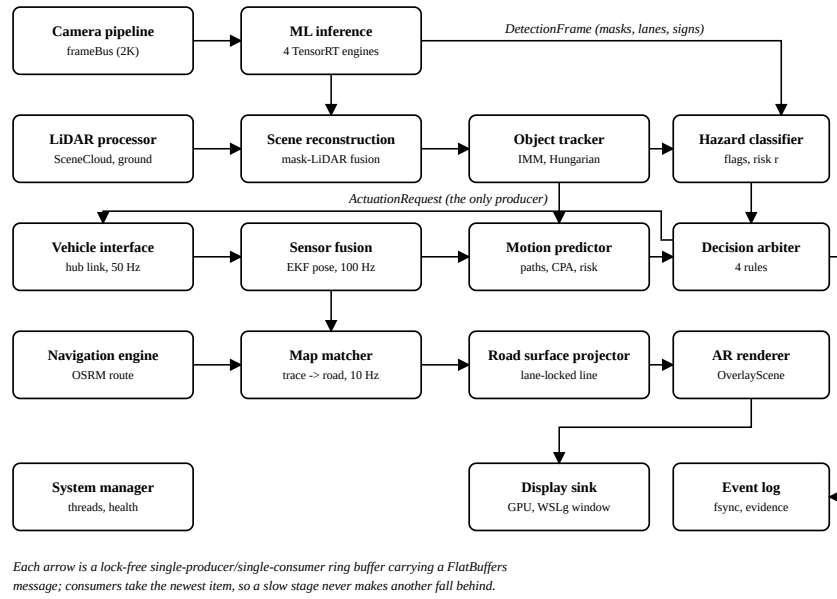


Figure 10: Host software dataflow; each arrow is a typed message on a ring buffer

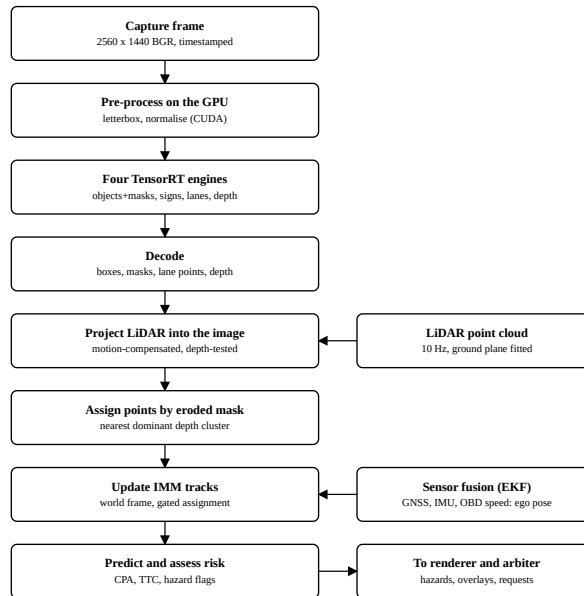
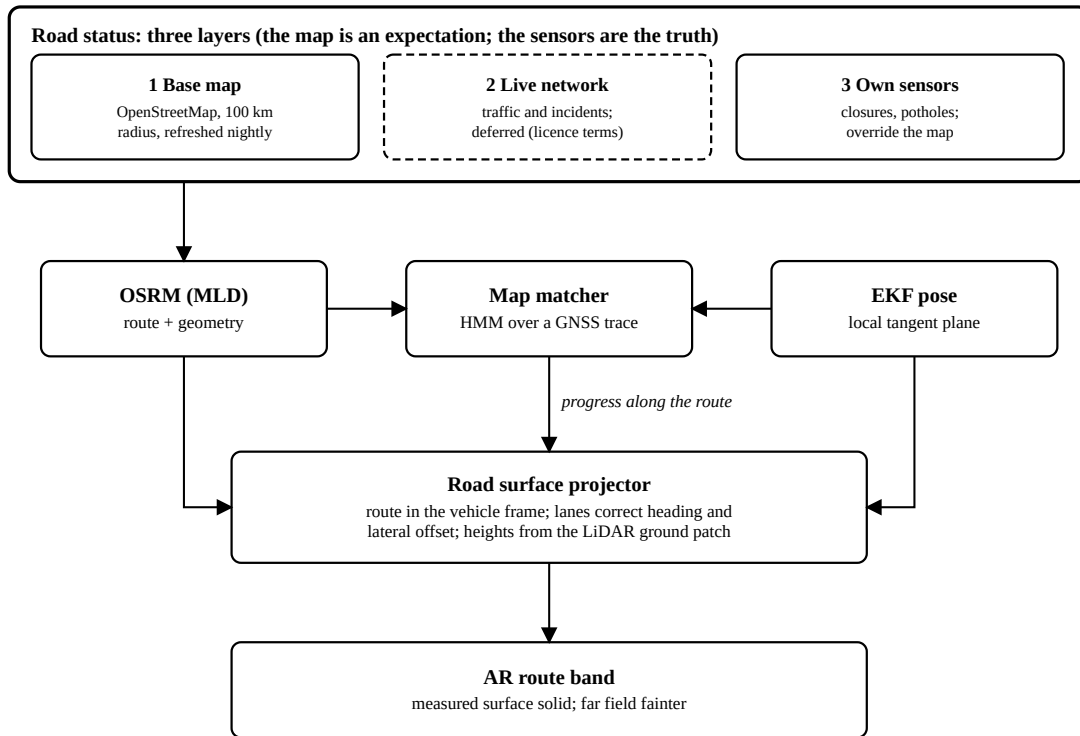


Figure 11: Per-frame perception and fusion flow



Measured on a recorded Nairobi drive (2,773 GNSS fixes): median snap distance 3.1 m; 98.7 % of fixes within 20 m.

Figure 12: Navigation pipeline and the three road-status layers

D.1 Decision arbiter, rule 1 (brake)

```

for each confirmed track o in the ego path, ahead of the bumper:
    if o.range was measured by LiDAR within 200 ms
        and o.closingSpeed > 0.5 m/s and all values finite:
            ttc(o) = o.gap / o.closingSpeed
target = argmin ttc
armed = hubReportAge ≤ 100 ms and killSwitch released
        and no actuator fault and eventLog healthy
if target exists and ttc(target) < 1.8 s and pedal not pressed and armed:
    intensity = min(configured 60, configured max 90, hub ceiling 90)
    log(rule 1, ttc, gap, closing speed, target, armed)    # before sending
    send BRAKE(intensity)
  
```

D.2 Mask-based fusion, per object

```
for each LiDAR point p (time t_p):
    q = compensate(p, t_p → t_camera, ego speed, yaw rate)
    drop q if |ground.distance(q)| < 0.15 m
    (u, v) = project(q) # pinhole + k1 k2 p1 p2 k3 distortion
    keep only the nearest point in each 4 px cell # depth test
for each detection d:
    P = visible points inside erode(mask(d), 1 cell)
    clusters = split P by depth at gaps > 0.5 m
    c = nearest cluster with ≥ 20% of P
    if |c| ≥ 3: range(d) = median(c), source = LIDAR
    else: range(d) = MiDaS estimate scaled by LiDAR objects, source =
ESTIMATED
non-ground corridor points claimed by no mask → UNKNOWN obstacles
```

D.3 Hub watchdog task (every 20 ms)

```
kick IWDG (500 ms)
if now - lastValidHostFrame > 200 ms or report loop silent > 200 ms
    or killSense pressed or frameErrors > 10/50:
        release actuator directly (PWM 0, enables low, magnet off) # no mutex
        relays off; forget last command; state = DISARMED
```

D.4 HMM map matching

For fix z_t and candidate road point x_t , emission $p(z_t|x_t) \propto \exp(-\frac{1}{2}(|z_t - x_t|/\sigma)^2)$; transition $p(x_t|x_{t-1}) \propto \exp(-|d_{\text{route}} - d_{\text{straight}}|/\beta)$, where d_{route} is the driving distance between candidates and d_{straight} the distance between fixes; the Viterbi path over a sliding window gives the matched positions [28].

Appendix E: Test Plan and Checklists

Stage A: actuator bench rig (gate)

- Rig check: actuator and pedal fixture on the bench, not in a vehicle.
- Intensity sweep: applied intensity, current, force and response time logged per step.
- Push-back: a person moves the pedal by foot at maximum configured intensity.
- Kill switch pressed mid-actuation: drive and current fall to zero; actuator goes limp.
- USB unplugged mid-actuation: release and zero current within 200 ms, measured by the hub.
- Inter-box cable pulled mid-actuation: limp at once; fault 5 reported; no false overcurrent latch.
- Exit: all six pass on three consecutive attempts, and again on the custom PCB hardware.

Stage B: stationary in vehicle (handbrake on, wheels chocked)

- SensorReports show real OBD-II (read over CAN), GNSS and IMU data.
- IMU mount and camera pitch calibrated; the knock check reads under 1°.
- Inter-box cable pulled with the actuator applied: releases, fault 5.
- A person walking into view at close range produces the correct request in the event log.
- The actuator engages and the driver can push through it, as on the bench.
- Kill switch and watchdog checks repeated in the vehicle.
- Road-projection check performed on first movement.

Stage C: empty field, spotter present

- Actuation, override and watchdog behaviour hold at low vehicle speed.
- Recorded explicitly: stopping-distance and TTC calibration are *not* validated on loose ground.

Stage D: closed campus road section

- Formal closure arranged with campus security; department safety notification filed in advance.
- Near-zero foot traffic; spotters at both ends and one alongside; agreed abort signal.
- Speed capped at 5–10 km/h.
- First runs with a manual confirm-to-brake step before automatic triggering.
- Full event log retained from every run.

Appendix F: Development Status and Verification to Date

The software is written in C++17 (host) and C++ on FreeRTOS (hub), built with CMake and PlatformIO, and kept under version control with a progress log and decision log. Table 11 lists the verification carried out to date.

Table 11: Verification carried out to date (September 2026)

Area	Verification	Result
Host unit tests	All modules, with address and undefined-behaviour sanitizers	232/232 pass
Routing and GPU tests	OSRM routing, map matching, GPU renderer	29/29 pass
Hub firmware tests	Protocol, CRC, framing, safety core, OBD-over-CAN poller (native build); 600,000-event fuzz; mutation checks	38/38 pass
Hub firmware build	STM32F405 target (CAN, bench and ELM327 variants)	71 KB flash; 7.3% RAM
Inference	Four networks together, recorded 2K video	Median 17.8 ms / frame
Sign detector	Held-out validation; Kenyan audit	mAP50 0.716; 24/25
Decision arbiter	End-to-end approach simulation; 200,000-input fuzz	Brake at TTC 1.70 s; no ceiling violation
Navigation	CBD–Thika route; closure reroute; recorded drive	43.7 km; 1.2 s; median snap 3.1 m
Renderer	2560×1440 benchmark; projection test	54–82 fps; within 2 px
Bench tooling	Protocol self-test of the bench logger	Pass; ready for hardware

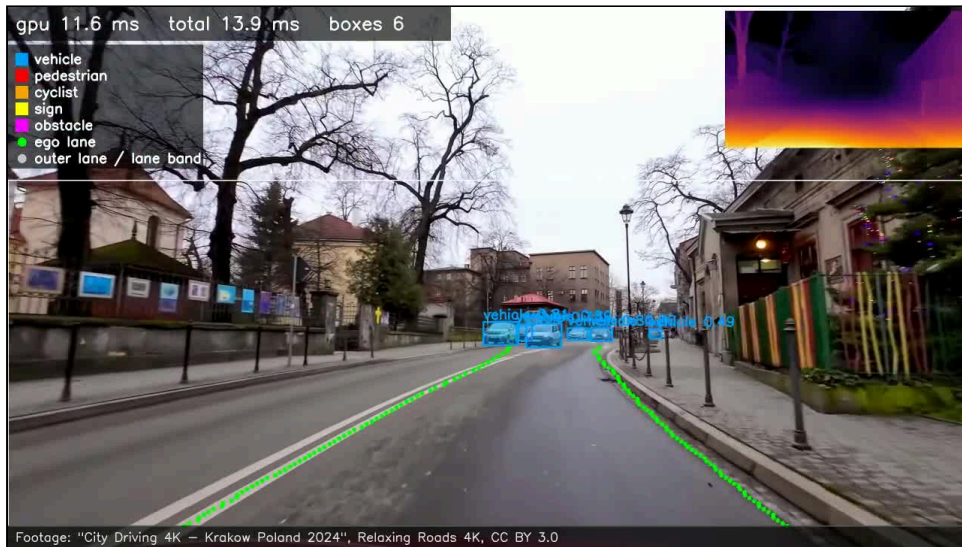


Figure 13: Ego-lane detection by UFLDV2 on recorded road footage (Relaxing Roads 4K, CC BY 3.0)



Figure 14: Kenyan audit image, ICRAF Road, Nairobi: road hump (0.77) and 30 km/h speed limit (0.78) detected (image: Matti Blume, CC BY-SA 4.0, Wikimedia Commons)



Figure 15: The sign detector on a Kenyan road (A104, Limuru–Naivasha): stop sign detected at confidence 0.93 (image: KartaView, user ToffeHoff, CC BY-SA 4.0)

Remaining work requires hardware: hub bring-up (phase 2), camera (4), LiDAR and calibration (6), the actuator bench gate (12), bench integration (13), installation (14) and staged validation (15).